Overview

- Goals:
  - Introduction to basic concepts of Python (and programming languages in general).
  - Programs existed before machines were built to execute them.
- ⚠ Does not attempt to replace any of the actual introductory lectures which will use Java.
- Organization: from 10-12 and 14-16 (+) on Mon, 14.09, 21.09 (TBD: 28.09).
- Requirements:
  - `https://colab.research.google.com/` (Google account for upload) or
  - `https://jupyter.org/try` (no account, but also no "AI support") or
  - local installation of Python with some IDE like vscodium with jupyter extension.

    (For windows: `winget install "python 3.14"; winget install vscodium; pip install jupyter; pip install sympy; pip install networkx; pip install matplotlib`; no support.)
  - ⚠ Provided code uses 2 spaces for indentation.

For personal use only, no commercial use – TUM CIT Department of Computer Science

1. Elementary algorithms and arithmetic
2. Polynomials and rational functions
3. Euclidean geometry in the plane
4. Solutions

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Elementary algorithms and arithmetic

Intro

Computing with natural numbers

Computing with integers

Computing with rational numbers

Primes*

Numbers in programming languages*

For personal use only, no commercial use – TUM CIT Department of Computer Science

Programming and algorithms

- Basic meaning of "algorithm", "program", ... is "plan" (see end of section).
- The word "algorithm" goes back to Al-Khwarizmi (ca. 780-850) and originally refered to "recipes" for arithmetic and solving certain quadratic equations.

  (But "algorithms" have existed already before 780, see the Euclidean algorithm and the sieve of Eratothenes later.)
- First step of "programming" is coming up with a "plan" to solve a problem.

  Second is to describe the plan in sufficient detail for some-one/-thing else.

  (But sometimes already a sufficiently precise problem description suffices.)
- A plan for a machine must (had to) be more detailed than for a human (in particular, if the machine interacts with the physical world) as the machine (ideally) does exactly as told but only has a "basic vocabulary" ("don't hurt human" won't work).
- A plan for a machine thus usually requires the use of a "programming language" which is more formal (stricter syntax, semantics) than natural language.
- Usually, we first come up with a plan which is sufficiently detailed for a (sufficiently educated) human and then break it down into "smaller steps" which a specific machine can execute (see example on next slide).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Example: an algorithm for solving certain quadratic equations

EG Problem: Given $a \in \mathbb{N}$, $b, c \in \mathbb{Z}$, determine all $x \in \mathbb{Q}$ s.t. $ax^2 + bx + c = 0$.

Compute $D := b^2 - 4ac$!

Let $k := 0$ and $s := 0$!

While $s < D$ do: set $k := k + 1$, then $s := s + 2k - 1$! (go back to start of line)

If $s = D$, compute solutions $\frac{-b-k}{2a}$ and $\frac{-b+k}{2a}$, else no solutions in $\mathbb{Z}$!

- "Pseudocode (mathematical cooking recipe language)" is often used for algorithms which is a sequence of conditional commands built from **if-do, while-do, do-until** and standard mathematical notation.

  The algorithm is then "translated" into a programming language (next).

  (The actual "coding". Modern LLMs will be able to directly translate the above pseudocode into Python code; see later.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Example: an algorithm for solving certain quadratic equations

EG Problem: Given $a \in \mathbb{N}$, $b, c \in \mathbb{Z}$, determine all $x \in \mathbb{Q}$ s.t. $ax^2 + bx + c = 0$.

Compute $D := b^2 - 4ac$!

Let $k := 0$ and $s := 0$!

While $s < D$ do: set $k := k + 1$, then $s := s + 2k - 1$! (go back to start of line)

If $s = D$, compute solutions $\frac{-b-k}{2a}$ and $\frac{-b+k}{2a}$, else no solutions in $\mathbb{Z}$!

▷ Translated to Python: ("reading" is relatively simple; "writing" requires training)

```
from fractions import Fraction          # tell Python to look up what a 'Fraction' is
a = 12; b = -12; c = -45                # let a = 12, b = −12, c = −45
assert a > 0                            # continue onlf if a > 0 else complain and stop
assert all(isinstance(x,int) for x in [a,b,c]) # assert that a, b, c ∈ ℤ
D = b**2 - 4*a*c                        # let D := b² − 4ac
k = 0; s = 0                            # start at k := 0, s := 0
while s < D: k += 1; s += 2*k-1         # while k² < D set k := k + 1, then (!) s := s + 2k − 1 = k² − (k − 1)²
if s == D:                              # s = k² = D and k ≥ 0, thus k = √D
  sol = {                               # set of solutions:
    Fraction(-b-k,2*a),                 # (−b−k)/2a
    Fraction(-b+k,2*a)                  # (−b+k)/2a
  }                                     # (a set does not know multiplicities)
else:
  sol = {}                              # there is no rational solution
print(sol)                              # internal representation
print({str(x) for x in sol})            # pretty printed
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

Calculators and computers

- "Computer" and "calculator" originally referred to humans. The first "mechanical calculators" (see Pascaline by Blaise Pascal) could only apply the "low-level" algorithms for basic arithmetic ($+, -$; already some relief) but a human still has to "store and execute" the high-level algorithm step by step.
- Algorithms do not require machines; but we want machines that can execute them for us as repeatedly applying an algorithm gets boring (which leads to errors).

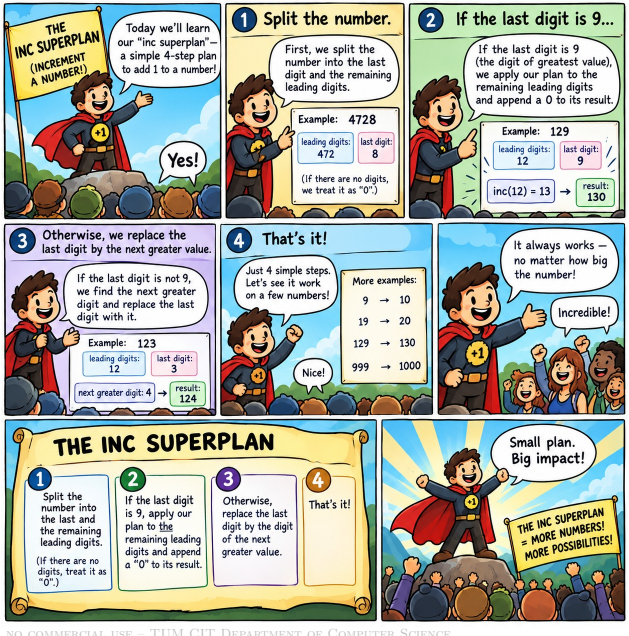
  (The Euclidean algorithm (300BC) is older than the word "algorithm". Shor's algorithm was published in 1994 but quantum computers are still in development.)
- Naturally, the wish arises that the machine can also remember the high-level algorithm which led to the analytical engine (1837; never built) by Charles Babbage and then to the Z3 (1941) by Konrad Zuse and the ENIAC (1945).
- Although the analytical engine was never built, Ada Lovelace realized that "[The Analytical Engine] might act upon other things besides numbers [...]".
- Modern computers still use numbers (resp. strings of digits) to represent arbitrary data and use arithmetic and logic operations to rewrite them step by step.
- We start a very important algorithm: incrementing a number given as a string (word, sequence) of digits! (Back to elementary school! Awesome!?)

For personal use only, no commercial use · TUM CIT Department of Computer Science

1 Elementary algorithms and arithmetic
Intro
Computing with natural numbers
Computing with integers
Computing with rational numbers
Primes*
Numbers in programming languages*

For personal use only, no commercial use – TUM CIT Department of Computer Science

The (awesome) superplan `incN` for (young) humans

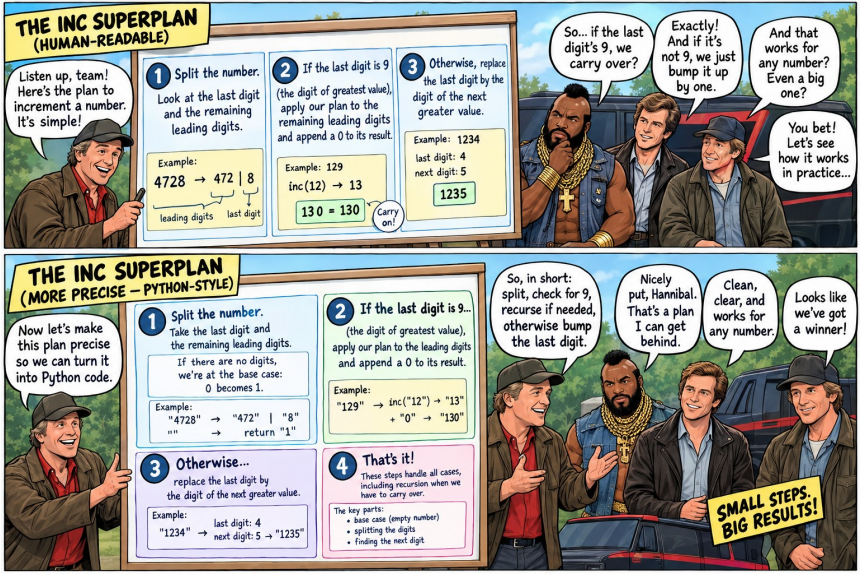


For personal use only, no commercial use – TUM CIT Department of Computer Science



For personal use only, no commercial use – TUM CIT Department of Computer Science

The (awesome) superplan `incN` for old humans



(Probably, I ran out of free "tokens" here ...)

For personal use only, no commercial use – TUM CIT Department of Computer Science

The "modern" way of programming

- Programming is planning/problem solving; depending on the available tools and concepts, the plan can be more natural for a human or more natural for a machine. Computer science thus knows different "programming paradigms" which emphasize different aspects.
- E.g. in declarative programming (logic programming, functional programming, constraint programming) we describe the problem (sufficiently precisely) to the computer but not how to solve it. Historically, this required a formal language (logic) and decision/search procedures, but modern LLMs can handle also problem descriptions in natural language – at least to some extent.
- Usually, some sort of counterexample guided "dialogue" is required to refine the plan or problem description until it is understood as intended (by the LLM or another human); but the refinement can usually stop on a higher abstraction than when using a classic programming language.

  (But in the end, it is undecidable whether a program satisfies a specification.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

The "modern" way of programming

Exc Ask Gemini: (Actually, do not do this, simply watch the online demonstration.)

- ▷ Can you translate the following into Python? [insert the human-readable "superplan" from the comic].
- ▷ Can you generalize this so that it takes an additional parameter 'digits' which is assumed to list the admissible digits in increasing order?

⚠ In general, you should not trust code from someone else (including an LLM).

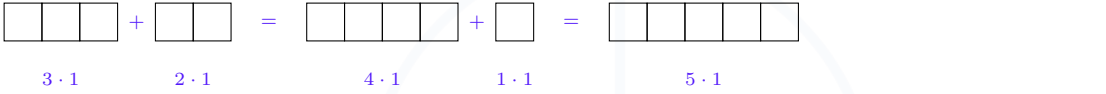
I.e., you still need to be able to read and understand the code — which is sometimes harder than writting it yourself as the "code" often skips the arguments why some "rule" is used in the code/plan (syntax vs semantics).

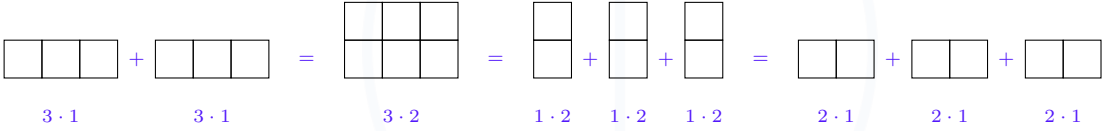
- ▷ You can use comments to explain your reasoning (sketch the proof/argument) at least informally — while Python does not read your comments, it might help other humans (an LLM, and also yourself at a later point in time) to understand what your program is supposed to do.
- ▷ In general, we need to prove/argue that a program is indeed solving the problem (satisfying the specification): to this end, we can annotate the code with formal statements (e.g. in Floyd-Hoare logic) to reason from (strongest) post- to (weakest) precondition (or the other way around; see predicate transformer).

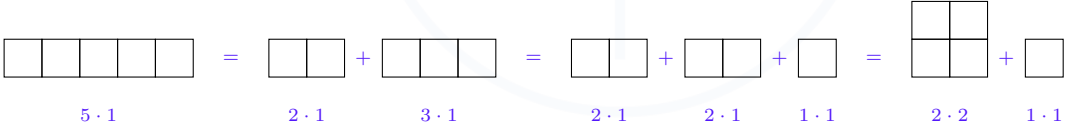
For personal use only, no commercial use – TUM CIT Department of Computer Science

$\mathbb{N}$ aka Tetris

- The next step is to use `incN` to tell Python how to also add, subtract, multiply and divide natural numbers just as it is done in elementary school.







For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

Exc Implement the following functions using:

- `decN(a)`: decrements `a` (`incN` reversed); if `a` represents 0, return `None`.
- `iszeroN(a)`: returns if a string `a` represents 0 or not.
- `cmpN(a,b)`: returns `"<",">","="` depending which is true.
- `addN(a,b)`, `subN(a,b)`, `mulN(a,b)`: addition, subtraction, and multiplication of natural numbers represented as strings.
- `divmodN(a,b)`: given $a \in \mathbb{N}_0$ and $b \in \mathbb{N}$ it should return $k, r \in \mathbb{N}_0$ with $a = kb + r$ and $r < b$ (Euclidean division/division with remainder).
- (*) `convertN(a,digits1,digits2)`: convert from `digits1` to `digits2`.

  (Internally, numbers are represented w.r.t. base 2 (binary). If you use `print` or `str`, then Pyhton converts the number into a string of digits w.r.t. base 10.)

Efficiency does not matter, keep it simple, use `incN` and `decN`.

⚠ Colab will try to complete most code for you: this means that you can learn the syntax "on-the-fly" but you still need to read and understand what Colab proposes (sometimes this can also be misleading and/or incorrect. You can also try to ask Gemini to solve the problems for you and try to check its answer).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise: Gemini inside of Colab

Exc On Colab you can directly access Gemini. Create a new code cell and click "generate", then input the "query":

"Gemini, please generate Python code which solves the quadratic equation 'axx+bx+c=0' assuming that 'a,b,c' are of type 'int' and that only exact rational solutions should be returned." (Always be polite to the LLM!)

- ▷ Compare the result to the initial example. (Gemini within Colab will read everything written so far including comments, and thus might re-use already existing code.)
- ▷ We still need to be able to understand the output and check if it is really doing what we want – which is still non-trivial and requires experience.

  In particular, the more we know, the better we can tell the LLM what we want.
- ▷ Colab has the "magic wand" on a code box which allows to ask Gemini to explain the code. This can help sometimes but in general there is no program which can "explain" or "understand" every program.

(LLMs have been trained to "understand" natural language by (partially) mimicking how we assume that our brain works. So we can give an LLM a plan or just a problem description in natural language and hope it understood what we tried to say (and deems to do so) ... just as with humans.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Elementary algorithms and arithmetic
Intro
Computing with natural numbers
Computing with integers
Computing with rational numbers
Primes*
Numbers in programming languages*

For personal use only, no commercial use – TUM CIT Department of Computer Science

Integers

- So far we can only compute with natural numbers (represented as strings of digits).
- In order to extend it to integers, we could explicitly introduce the sign symbols $+, -$ and add corresponding rules to `inc,dec,addN,mulN,...`

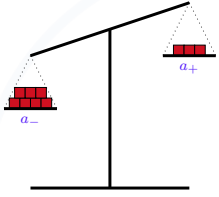
  But this would mean to change existing code which is boring and might introduce (additional) errors.
- Alternatively, we can do what mathematicians do:

  Built upon that what is already (well-)defined, i.e. use it as subprocedures.

  (Or we use Pythons `Fraction` but that would be too simple.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

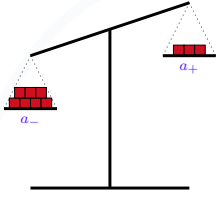
- So far we can only compute with natural numbers (represented as strings of digits).
- An integer $a \in \mathbb{Z}$ can be explained by a pair $(a_+, a_-) \in \mathbb{N}_0^2$ of natural numbers:



- The two natural numbers a_+, a_- represent the number of unit masses on right and left side of a two-sided scale, respectively (annoyingly, the order changes):
  - If $a_+ > a_-$, then the scales tend to the right; the signum of (a_+, a_-) is $+1$.
  - If $a_+ < a_-$, then the scales tend to the left; the signum of (a_+, a_-) is -1.
  - If $a_+ = a_-$, then the scales are balanced (al-jabr); the signum of (a_+, a_-) is 0.
  - Flipping the sign, simply means to swap left and right.
  - The absolute value $|(a_+, a_-)|$ is $\max(a_+, a_-) - \min(a_+, a_-)$, i.e. bring the smaller weight to the left scale.

For personal use only, no commercial use – TUM CIT Department of Computer Science

- So far we can only compute with natural numbers (represented as strings of digits).
- An integer $a \in \mathbb{Z}$ can be explained by a pair $(a_+, a_-) \in \mathbb{N}_0^2$ of natural numbers:



- We want to define $(a_+, a_-) + (b_+, b_-)$ without using subtraction or a sign, so that the result still represents $(a_+ - a_-) + (b_+ - b_-)$. Hence, we set

$$(a_+, a_-) + (b_+, b_-) := (a_+ + b_+, a_- + b_-)$$

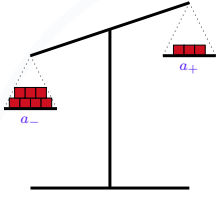
(I.e. add the weights on both sides independently.)

- Similarly, we want that the result of $(a_+, a_-) \cdot (b_+, b_-)$ represents $(a_+ - a_-) \cdot (b_+ - b_-)$. Thus, we set

$$(a_+, a_-) \cdot (b_+, b_-) := (a_+b_+ + a_-b_-, a_+b_- + a_-b+)$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

- So far we can only compute with natural numbers (represented as strings of digits).
- An integer $a \in \mathbb{Z}$ can be explained by a pair $(a_+, a_-) \in \mathbb{N}_0^2$ of natural numbers:



- ⚠ $-(a_+, a_-) := (a_-, a_+)$ (simply swap left/right).
- $(a_+, a_-) - (b_+, b_-) := (a_+, a_-) + (b_-, b_+)$.
- $(a_+, a_-) \equiv (0, 0)$ **iff** $a_+ = a_-$ (balanced).
- $(a_+, a_-) \equiv (b_+, b_-)$ **iff** $(a_+, a_-) - (b_+, b_-) \equiv (0, 0)$ **iff** $a_+ + b_- = a_- + b_+$.

  ("$\equiv$" means here "equivalent as integers", "representing the same integer".)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

```
type N = str           # types are in the first place names for collections of objects
type Z = tuple[N,N]    # an integer is a pair of natural numbers (a+, a−)

def addZ(a: Z, b: Z) -> Z:  # a[0] is a+ and a[1] is a−
  # (a+, a−) +Z (b+, b−) := (a+ +N b+, a− +N b−)
  return addN(a[0],b[0]), addN(a[1],b[1])

def mulZ(a: Z, b: Z) -> Z:
  # (a+, a−) · (b+, b−) := (a+b+ + a−b−, a+b− + a−b+)
  return addN(mulN(a[0],b[0]), mulN(a[1],b[1])), addN(mulN(a[0],b[1]), mulN(a[1],b[0]))
```

⚠ `addZ` and `mulZ` simply rewrite the LHS to the defining RHS.

Exc Implement a function `negZ` which returns the result of $-(a_+, a_-)$.

Exc Implement a function `subZ` which returns the result of $(a_+, a_-) - (b_+, b_-)$.

Exc Formally (and also to Python) $(3, 5)$ and $(0, 2)$ are not the same, but only equivalent when read as integer.

Implement a function `reduceZ` which uses `decN` and `iszeroN` to reduce an integer (a_+, a_-) to its normal form where at least one component is zero.

Exc Implement a function `signabsZ` which returns a pair consisting of the signum ("+", "-", "0") and the absolute value $|(a_+, a_-)|$ (a natural number as a string).

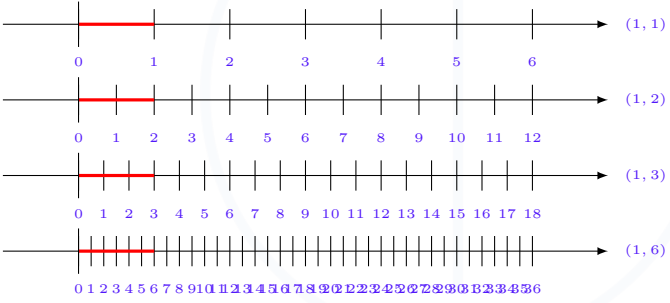
For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Elementary algorithms and arithmetic
- Intro
- Computing with natural numbers
- Computing with integers
- Computing with rational numbers
- Primes*
- Numbers in programming languages*

For personal use only, no commercial use – TUM CIT Department of Computer Science

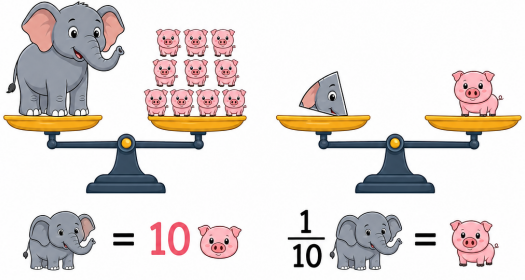
Rational numbers

- The next step is to implement the rational numbers using our existing code.
- Again, we stick to the standard definition used in mathematics:
  - Typically, we write $^n/_d$ or $\frac{n}{d}$ for the pair (n, d) but this is just notation.
  - Conceptually, $\frac{1}{d}$ means that we break the unit 1 into d equal parts in order to obtain a finer measure. Given $\frac{1}{d_1}$ and $\frac{1}{d_2}$, there is a common refinement $\frac{1}{d_1 d_2}$.



- Addition: $(a, b) + (c, d) := (ad, db) + (bc, bd) := (ad + bc, bd)$ (same measure).
- Multiplication: $(a, b) \cdot (c, d) := (ac, b) \cdot (1, d) := (ac, bd)$.

For personal use only, no commercial use – TUM CIT Department of Computer Science



(All hail D(h)ALL-E the AI ... and who needs drinking water anyways? We have Brawndo!)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises


```
type Q = tuple[Z,Z]   # a rational number n/d is a pair of integers (n, d).

def addQ(a: Q, b: Q, digits=None) -> Q:
  return addZ(mulZ(a[0], b[1], digits), mulZ(a[1], b[0], digits)), mulZ(a[1], b[1], digits)

def mulQ(a: Q, b: Q, digits=None) -> Z:
  # this does not check if we divide by 0
  return mulZ(a[0], b[0], digits), mulZ(a[1], b[1], digits)
```

- ▷ `addQ` and `mulQ` again simply rewrite the LHS to the defining RHS.

Exc Implement the following functions:

- ▷ `iszeroQ`: checks if the rational number is zero.
- ▷ `negQ` and `invQ`: return the additive and multiplicative inverse of a rational number, respectively.
- ▷ `subQ` and `divQ`: subtraction and division of rational numbers.

(Or check if the problem statements are sufficiently precise for an LLM to solve the problems for you and then try to understand the answer)

- ▷ If you like, consider arbitrary bases, else stick to base 10.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Greatest common divisor: normalizing rational numbers

- In our implementation, we do not cancel the numerator and denominator by the greatest common divisor yet. To this end, we use the Euclidean algorithm.
- It is based on the observation that for every $d \in \mathbb{N}$ we have:

  "d divides both a and b **iff** d divides both $b - a$ and a."

  Short: $\gcd(a, b) = \gcd(a, b - a)$.
- The typical convention is that $\gcd(0, a) = \gcd(a, 0) = a$ (s.t. $\gcd(0, 0) = 0$).
- We thus obtain the recursion: to compute $\gcd(a, b)$ for $a, b \in \mathbb{N}_0$
  - IB if $\min(a, b) = 0$: return $\gcd(a, b) = \max(a, b)$
  - IS else: return $\gcd(a, b) = \gcd(\min(a, b), \max(a, b) - \min(a, b))$.
    (Note that we get "smaller": $a + b$ decreases to $\max(a, b)$.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Greatest common divisor: normalizing rational numbers

- In our implementation, we do not cancel the numerator and denominator by the greatest common divisor yet. To this end, we use the Euclidean algorithm.
- It is based on the observation that for every $d \in \mathbb{N}$ we have:

  "d divides both a and b **iff** d divides both $b - a$ and a."

  Short: $\gcd(a, b) = \gcd(a, b - a)$.
- The typical convention is that $\gcd(0, a) = \gcd(a, 0) = a$ (s.t. $\gcd(0, 0) = 0$).
- We thus obtain the recursion: to compute $\gcd(a, b)$ for $a, b \in \mathbb{N}_0$
  - IB if $\min(a, b) = 0$: return $\gcd(a, b) = \max(a, b)$
  - IS else: return $\gcd(a, b) = \gcd(\min(a, b), \max(a, b) - \min(a, b))$.
    (Note that we get "smaller": $a + b$ decreases to $\max(a, b)$.)
- This can be simplified to (why?)
  - IB if $a = 0$: return $\gcd(a, b) = b$
  - IS else: return $\gcd(a, b) = \gcd(b \bmod a, a)$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

Exc Implement a function `gcdN` which uses the Euclidean algorithm to compute the greatest common divisor for a pair of natural numbers.

Exc Implement a function `reduceQ` which normalizes a rational number so that the denominator is a positive natural number, and the greatest common divisor of the (absolute value of the) numerator and the denominator is 1.

⚠ In principle, we can also first compute the prime factorization of both the numerator and denominator and then cancel prime factors until both numerator and denominator do not share any common prime divisors anymore (usually what you do in school).

Computing the prime factorization of a natural number n is in general conjectured to be "computationally intractable".

This makes the Euclidean algorithm so important as it avoids the prime factorization.

(See the optional section on primes for one of the oldest known algorithm for computing all primes up to some given $n \in \mathbb{N}$.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Elementary algorithms and arithmetic
Intro
Computing with natural numbers
Computing with integers
Computing with rational numbers
Primes*
Numbers in programming languages*

For personal use only, no commercial use – TUM CIT Department of Computer Science

Primes and the sieve of Eratosthenes

- Besides algorithms for basic arithmetic, the Euclidean algorithm and the sieve of Eratosthenes are considered examples of ancient algorithms.

Def Recall that $n \in \mathbb{Z}$ is a prime if both $n > 1$ and no $d \in \{2, \dots, n-1\}$ divides n.

EG In Python we can implement this more or less directly (but inefficienty):

```
def isprime(n: int) -> bool:
  return n > 1 and all(n % d != 0 for d in range(2,n))

print([n for n in range(100) if isprime(n)])
```

- A famous problem is the asymptotic growth of the prime-counting function $\pi(x) := |\{p \in \mathbb{N} \mid p \leq x, p \in \mathbb{P}\}$ (not to be confused with $\pi = 3.141...$).

  One can show that $\pi(x)$ grows asymptotically as $\frac{x}{\log(x)}$ which is called prime number theorem. (See the following code for an example.)

- Calling `isprime` for all $n \leq x$ in order to determine $\pi(x)$ is very inefficient:

  The sieve of Eratosthenes instead discovers the primes in $[0, x]$ in increasing order. Initially, all integers in $[2, x]$ are considered as potentially prime. Whenever the next smallest prime p (starting with $p = 2$) is discovered all of its multiples $p\mathbb{Z}$ in $[2, x]$ are marked as non-prime.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Primes and the sieve of Eratosthenes, plotting using matplotlib


```
import matplotlib.pyplot as plt
import math
from itertools import accumulate

def plot_pi(n: int):
  # Sieve of Eratosthenes
  is_prime = [True for _ in range(n+1)]
  is_prime[0] = is_prime[1] = False
  for k in range(2,n+1):
    if is_prime[k] is True:          # all multiplies of some k' < k have already been marked as 'False'.
      for j in range(k*k, n+1, k):   # mark all multiples of k between k^2 and n
        is_prime[j] = False

  # pi[i]: number of primes in {0,...,i}
  pi = list(accumulate(is_prime))

  # absolute w.r.t. n/log(n)  (natural logarithm)
  plt.plot(list(range(n+1)),pi)
  plt.plot(list(range(n+1)),[i/max(1,math.log(max(i,1))) for i in range(n+1)])
  plt.show()

  # relative w.r.t. n/log(n)  (natural logarithm)
  plt.plot(list(range(1,n+1)),[pi[i]/(i/max(1,math.log(max(i,1)))) for i in range(1,n+1)])
  plt.show()
  return

plot_pi(1000)
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise: prime factorization

- Every $n \in \mathbb{N}$ can be written as a finite product of powers of distinct primes:

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

  A usually convention is to sort the primes s.t. $p_1 < p_2 < \ldots < p_k$.

- If $m = p_1^{e_1} \cdots p_i^{e_i}$ is known, only $\frac{n}{m}$ needs to be factorized and only primes $p > p_i$ need to be considered.

- Show:
  - The least $d > p_i$ which divides m must be p_{i+1} and odd.
  - If d is the next odd number greater p_i and $d^2 > m$, then m is prime.

Exc Write a program that outputs the sequence $((p_1, e_1), (p_2, e_2), \ldots, (p_k, e_k))$ given a positive interger n.

For personal use only, no commercial use – TUM CIT Department of Computer Science

1. Elementary algorithms and arithmetic
   - Intro
   - Computing with natural numbers
   - Computing with integers
   - Computing with rational numbers
   - Primes*
   - Numbers in programming languages*

For personal use only, no commercial use – TUM CIT Department of Computer Science

Numbers in Python

```
# Python knows 'int' for ℤ and 'Fraction' for ℚ

a = 2 ** 31 - 1
print(a)    # 2147483647
print(2*a)  # 4294967294
print(3*a)  # 6442450941

from fractions import Fraction
a = Fraction(1,10)
b = Fraction(3,10)
print(a)    # 1/10
print(2*a)  # 1/5
print(3*a)  # 3/10
print(b)    # 3/10

# in C,C++, Java 'int' and 'float' can only represent finite subsets of ℤ and ℚ

import numpy as np
a = np.array([2 ** 31 - 1], dtype=np.int32)  # only {−2^31, ..., 2^31 − 1}
print(a)    # [2147483647]
print(2*a)  # [-2]
print(3*a)  # [2147483645]

a = np.array([0.1], dtype=np.float16)  # only ±1.d_1 ... d_10 · 2^(e−15) with e < 32
b = np.array([0.3], dtype=np.float16)  # so actually only a finite subset of ℚ
print(a)    # [0.1]
print(2*a)  # [0.2]
print(3*a)  # [0.2998]                 # unexpected results due to finite precision/rounding
print(b)    # [0.3]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

Exc Implement a Python function which takes $p, q, b, l \in \mathbb{N}_0$ as input (as Python `int`) and outputs the first l digits of the representation of $\frac{p}{q}$ w.r.t. base b.

You may assume that $0 \leq p < q$ and $1 < b \leq 10$.

Exc Use your code to compute the binary representation of $\frac{3}{4}$, $\frac{1}{10}$, $\frac{2}{10}$ and $\frac{3}{10}$. Try to understand, why `3*0.1==0.3` is false in Python using `floats`.

Exc (*) Show: For every rational $\frac{p}{q}$ with $0 \leq p < q$ and every base $b > 1$, its representation $0.d_1d_2d_3\ldots$ is "ultimately periodic", i.e. there exists some i and l s.t. $d_i \ldots d_{i+l-1} = d_{i+kl} \ldots d_{i+kl+l-1}$ for all $k \in \mathbb{N}_0$ (e.g. $[\![0.3]\!]_{10} = [\![0.0\overline{1001}]\!]_2$).

Exc (*) Extend your implementation so that it also determines the periodic part of the representation which is repeated forever.

(Make your program "remember" what it has already solved; see the intro slides on calculi.)

Exc (*) Show that if $0.d_1d_2\ldots$ is ultimately periodic with $d_i \in \mathbb{Z}_b$, then $[\![0.d_1d_2\ldots]\!]_b$ is a rational number.

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Elementary algorithms and arithmetic

2 Polynomials and rational functions

3 Euclidean geometry in the plane

4 Solutions

For personal use only, no commercial use – TUM CIT Department of Computer Science

2 Polynomials and rational functions
Polynomial functions
Rational functions

For personal use only, no commercial use – TUM CIT Department of Computer Science

Univariate polynomials over $\mathbb{R}$

Def A (real) polynomial is a (formal) arithmetic expression built from variables, constants (in $\mathbb{R}$), addition, subtraction, and multiplication.

Using distributivity, commutativity and associativity, a polynomial can always be expanded into a finite sum of monomials (products of a real numbers and variables) .

EG $5X+10XY^2Z-YZ \qquad (X+Y)(X-Y) = X^2-Y^2 \qquad (X-2)^2 = X^2-4X+4$

A univariate polynomial uses only a single variable, e.g. X, and is of the form

$$\sum_{k=0}^{n} a_k X^k = a_0 + a_1 X + a_2 X^2 + \ldots + a_n X^n$$

with coefficients $a_k \in \mathbb{R}$.

$\mathbb{R}[X]$ denotes the set of all real polynomials (polynomials with real coefficients) in the single variable X.

⚠ Usually, the symbol X used as a variable is distinguished from the symbol x representing a concrete real number.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Univariate polynomials in Python

```
from typing import *
from numbers import Number
import math

type Coeffs = Sequence[Number]

def poly_str(a: Coeffs) -> str:
  return '+'.join(f'{ai}X^{i}' for i, ai in enumerate(a)) if len(a) else "0"

def poly_deg(a: Coeffs) -> int:
  d = len(a) - 1
  while d >= 0 and a[d] == 0: d -= 1
  return d

a = (-1,0,1,0)  # a = −1 · X^0 + 0 · X^1 + 1 · X^2 + 0 · X^3 = −1 + X^2, deg(a) = 2
print(f"a={poly_str(a)}, deg(a)={poly_deg(a)}")
```

- An univariate polynomial $a_0 + a_1X + \ldots + a_nX^n$ can be represented as the sequence $a = (a_0, a_1, \ldots, a_n)$ of its coefficients.
  - ⚠ $(a_0, a_1, \ldots, a_n)$ and $(a_0, a_1, \ldots, a_n, 0, ..., 0)$ represent the same polynomial.
  - In fact, the sequence of coefficients is $(a_0, a_1, \ldots, a_n, 0, 0, \ldots)$.
  - But of course it suffices to store $(a_0, a_1, \ldots, a_n)$ if $a_m = 0$ for all $m > n$.
  - Typically, we assume $a_n \neq 0$ (but this is not necessary).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Univariate polynomials in Python

```
from typing import *
from numbers import Number
import math

type Coeffs = Sequence[Number]

def poly_str(a: Coeffs) -> str:
  return '+'.join(f'{ai}X^{i}' for i, ai in enumerate(a)) if len(a) else "0"

def poly_deg(a: Coeffs) -> int:
  d = len(a) - 1
  while d >= 0 and a[d] == 0: d -= 1
  return d

a = (-1,0,1,0)  # a = −1 · X^0 + 0 · X^1 + 1 · X^2 + 0 · X^3 = −1 + X^2, deg(a) = 2
print(f"a={poly_str(a)}, deg(a)={poly_deg(a)}")
```

- An univariate polynomial $a_0 + a_1X + \ldots + a_nX^n$ can be represented as the sequence $a = (a_0, a_1, \ldots, a_n)$ of its coefficients.
- If we drop trailing zeros, then either $a_n \neq 0$ or $a = ()$ is the empty tuple.
  - If $a_n \neq 0$, then $\mathrm{LC}(a) = a_n$ is its leading coefficient and $\deg(a) = n$ is its degree.
  - If $a = ()$, then $a = 0$ is the zero polynomial;
    $\deg(0)$ is defined to be either $-\infty$ or -1 (depends on what is more convenient).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Poynomial functions: evaluation of polynomials

```
def poly_eval(a: Coeffs, x: Number) -> Number:
  return sum(ai*x**i for i, ai in enumerate(a))

import matplotlib.pyplot as plt
def poly_plot(a: Coeffs, xmin: Number, xmax: Number, nsteps: int=100):
  xs = [xmin+i*((xmax-xmin)/nsteps) for i in range(nsteps+1)]
  ys = [poly_eval(a,x) for x in xs]
  fig, ax = plt.subplots()
  ax.set_aspect('equal')
  ax.plot(xs,ys)
  plt.show()

poly_plot((-2,0,1), -2, 2)
```

- We can use `matplotlib` to plot the function $[x_{\min}, x_{\max}] \to \mathbb{R}, x \mapsto a(x)$:
  - ⚠ A polynomial is not a function, a function requires a domain.
  - `poly_eval` evaluates the polynomial a at a given number x, i.e. it replaces X by e.g. $x = 5$ and then computes $a(5) := a_0 + a_1 \cdot 5 + \ldots + a_n \cdot 5^n$.
  - As in school, we compute $(x_i, a(x_i))$ for $x_i = x_{\min} + \frac{x_{\max} - x_{\min}}{N} \cdot i$ and $i \in \{0, 1 \ldots, N\}$ and let `matplotlib` connect these points using line segments.
    (`matplotlib` wants the x_i and $a(x_i)$ as separate sequences like the rows of a table.)
  - As polynomial functions are continuous, the larger n, the closer the approximation will be to the actual function.

For personal use only, no commercial use · FAU CITI Department of Computer Science

Exercises

Exc As every polynomial function f is continuous, if $x_l < x_r$ and $f(x_l) < 0 < f(x_r)$ (or $f(x_l) > 0 > f(x_r)$), then there is some $z \in [x_l, x_r]$ with $f(z) = 0$.

The bisection method recursively searches for a zero within either $[x_l, \frac{x_l+x_r}{2}]$ or $[\frac{x_l+x_r}{2}, x_r]$ or stops depending on the sign of $f(\frac{x_l+x_r}{2})$ and $|x_r - x_l|$.

Use the bisection method to approximate $\sqrt{2}$ by searching for the only root of $X^2 - 2$ in $[0, 2]$; stop, if $|x_l - x_r| < 10^{-9}$.

(Bisection is a special case of "binary search".)

Exc Our naive implementation of `poly_eval` is not very efficient. Rewrite `poly_eval` so that it uses Horner's method:

$$a(x) = a_0 + x(a_1 + x(a_2 + x(\ldots(a_{n-1} + xa_n))))$$

(Or try "Gemini, please change `poly_eval` so that it uses Horner's method").

Exc Improve `poly_str` to avoid e.g. $+ - 5X^3$ or $0X^2$ in the output.

(Or try "Gemini, improve the current implementation `poly_str` s.t. ...").

For personal use only, no commercial use – TUM CIT Department of Computer Science

Addition and multiplication of polynomials

- Polynomials $p, q,$ can be added and multiplied e.g. by means of expanding the product and adding the coefficients of monomials of the same degree.

EG (addition can cancel the LC)

$$\begin{aligned}(1-X)(1+2X+3X^2) &= 1+2X+3X^2 \\ &\quad -X-2X^2-3X^3 \\ &= 1+X+X^2-3X^3\end{aligned}$$

$$\begin{aligned}(1-X^2)+(1+X)(X-1) &= (1-X^2)+(-1+X(1-1)+X^2) \\ &= (1-X^2)+(-1+X^2) \\ &= 0\end{aligned}$$

EG Alternatively using the "Cauchy product"

	1	$+2X$	$+3X^2$
1	1	$2X$	$3X^2$
$+X$	X	$2X^2$	$3X^3$
$+X^2$	X^2	$2X^3$	$3X^4$

$$(1+X+X^2)(1+2X+3X^2) = 1+3X+6X^2+5X^3+3X^4$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

Exc Implement functions `poly_add` and `poly_mul` which return the results of adding respectively multiplying two polynomials.

For `poly_add` use `zip_longest` from `itertools`:

- Recall that formally a polynomial is an infinite sequence but only with finitely many nonzero entries.
- `zip_longest(a,b,fillvalue=0)` takes two sequences $a = (a_0, \ldots, a_n)$ and $b = (b_0, \ldots, b_m)$ and combines them into the sequence $((a_0, b_0), (a_1, b_1), \ldots)$ where it uses 0 whenever a_i or b_i are not defined.

For `poly_mul` use the Cauchy product (or convolution)

$$c_k = a_0 b_k + a_1 b_{k-1} + \ldots + a_{k-1} b_1 + a_k b_0 = \sum_{i=0}^{k} a_i b_{k-i} = \sum_{i=\max(0,k-m)}^{\min(n,k)} a_i b_{k-i}$$

to compute the coefficients of the product

$$c_0 + c_1 X + c_2 X^2 + \ldots + c_{m+n} X^{m+n} = (a_0 + a_1 X + \ldots + a_n X^n) \cdot (b_0 + b_1 X + \ldots + b_m X^m)$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Polynomial division

▷ Euclidean division on the $\mathbb{Z}$ extends to polynomials division on $\mathbb{R}[X]$:

Def Compute s, r with $\deg(r) < \deg(b)$ s.t. $a = s \cdot b + r$ (assuming $\deg(b) \geq 0$)

- IB if $\deg(a) < \deg(b)$: return $s = 0, r = a$.
- IS else:
  compute $t := \frac{\mathrm{LC}(a)}{\mathrm{LC}(b)} X^{\deg(a)-\deg(b)}$
  compute $a' := a - b \cdot t$ (cancels the LC of a s.t. $\deg(a') < \deg(a)$)
  recurisvely compute s', r s.t. $a' = s' \cdot b + r$ (induction on $\deg(a')$)
  return $s := s' + t$ and r ($a = a' + b \cdot t = s' \cdot b + r + b \cdot t = (s' + t) \cdot b + r$)

⚠ s, r are unique ($(s - s')q = r' - r$ so $\deg(s - s') + \deg(q) = \deg(r - r') < \deg(q)$.)

EG $X^2 + 5X + 4 = \left(\frac{1}{2}X + \frac{3}{2}\right)(2X + 4) - 2$

$$
\begin{array}{rcrcr:l}
X^2 & + & 5X & + & 4 & 2X + 4 = \frac{1}{2}X + \frac{3}{2} \\
\cline{6-6}
-X^2 & - & 2X & & & \\
\cline{1-3}
 & & 3X & + & 4 & \\
 & - & 3X & - & 6 & \\
\cline{1-3}
 & & & - & 2 & \text{(remainder)}
\end{array}
$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Polynomial division

▷ Euclidean division on the $\mathbb{Z}$ extends to polynomials division on $\mathbb{R}[X]$:

Def Compute s, r with $\deg(r) < \deg(b)$ s.t. $a = s \cdot b + r$ (assuming $\deg(b) \geq 0$)

- IB if $\deg(a) < \deg(b)$: return $s = 0, r = a$.
- IS else:
  compute $t := \frac{\mathsf{LC}(a)}{\mathsf{LC}(b)} X^{\deg(a)-\deg(b)}$
  compute $a' := a - b \cdot t$ (cancels the LC of a s.t. $\deg(a') < \deg(a)$)
  recurisvely compute s', r s.t. $a' = s' \cdot b + r$ (induction on $\deg(a')$)
  return $s := s' + t$ and r ($a = a' + b \cdot t = s' \cdot b + r + b \cdot t = (s' + t) \cdot b + r$)

⚠ s, r are unique ($(s - s')q = r' - r$ so $\deg(s - s') + \deg(q) = \deg(r - r') < \deg(q)$.)

EG $X^2 + 5X + 4 = \left(\frac{1}{2}X + 2\right)(2X + 2)$

$$\begin{array}{rcrcrcl} X^2 & + & 5X & + & 4 & : & 2X + 2 = \frac{1}{2}X + 2 \\ -X^2 & - & X & & & & \\ \hline & & 4X & + & 4 & & \\ & - & 4X & - & 4 & & \\ \hline & & & & 0 & & \text{(remainder)} \end{array}$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Roots/zeros of polynomials

Def A concrete number $x_0 \in \mathbb{R}$ is a root/zero of $a = \sum a_i X^i$, if a evaluates to 0 at x_0 (i.e. $a(x_0) = 0$).

(The zero polynomial has infinitely many roots/zeros.)

- ▷ If $a(x_0) = 0$, then $(X - x_0)$ has to divide p without remainder. Thus:

Lem Every $a = \sum a_i X^i$ with $\deg(a) \geq 0$ can be factorized as

$$a = (X - x_1)^{m_1} \cdots (X - x_k)^{m_k} \cdot b$$

using polynomial division where

- ▷ b has no roots in $\mathbb{R}$,
- ▷ $x_1, \ldots, x_k \in \mathbb{R}$ are the pairwise distinct real roots of a,
- ▷ m_i is the multiplicity of x_i as root of a, and
- ▷ $m_1 + \ldots + m_k + \deg(b) = \deg(a)$ has to hold.

Cor $a = \sum a_i X^i$ with $\deg(a) \geq 0$ has at most $\deg(a)$ roots in $\mathbb{R}$.

EG

$$X^5 - X^4 - X + 1 = (X - 1)^2 (X + 1)^1 (X^2 + 1)$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

Exc Implement a function `poly_divmod` which on input a, b with $\deg(b) \geq 0$ computes polynomials s, r s.t. $a = sb + r$ and $\deg(r) < \deg(b)$.

Exc A GCD of two polynomials a, b is a polynomial d s.t. d divides both a and b (without remainder) and, if t divides both a and b, then t also divides d.

Implement a function `ploy_gcd` which computes a GCD of a, b.

(Recall the Euclidean algorithm: there is little to change, you only need `poly_mod`.)

EG $c(X + 1)$ is a GCD of $(X + 1)^2$ and $X^2 - 1$ for every $c \in \mathbb{R} \setminus \{0\}$.

⚠ The GCD of two polynomials is only unique up to the choice of its LC.

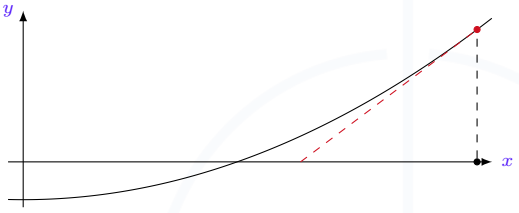
Usually, the LC is simply chosen to be 1.

If using `float`s, simply dividing by the LC can lead to numerical problems.

If you represent the LC using Python's `Fraction` and all other coefficients as `int` or `Fraction` such numerical problems can be avoided.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise: Newton's method (*)



$x_0 \approx 3, x_1 \approx 1.8334$

- Irrational roots of polynomials can be approximated using either the bisection method or preferably using Newton's method.

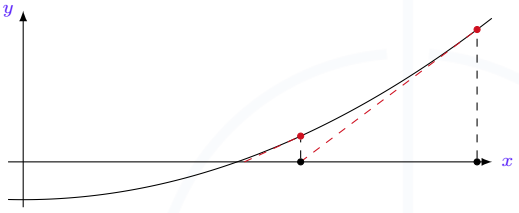
Def Newton's method for polynomials

- Input: $a = a_0 + a_1 X + \ldots + a_n X_n \in \mathbb{R}[X]$, initial approximation $x_0 \in \mathbb{R}$.
- For $k = 0, 1, \ldots$ iterate until $a(x_k) \approx 0$:
  1. Approximate a at x_k by the linear polynomial $l_k(X) := a'(x_k) \cdot (X - x_k) + a(x_k)$
  2. Choose x_{k+1} as the root of l_k: $x_{k+1} := x_k - \frac{a(x_k)}{a'(x_k)}$.

(In general, Newton's method will not converge; choice of x_0 matters.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise: Newton's method (*)



$x_1 \approx 1.834, x_2 \approx 1.46227$

- Irrational roots of polynomials can be approximated using either the bisection method or preferably using Newton's method.

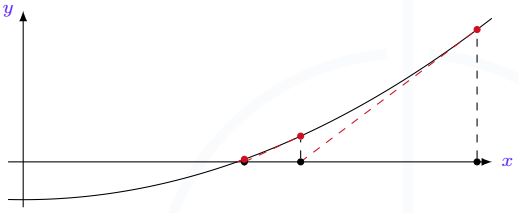
Def Newton's method for polynomials

- Input: $a = a_0 + a_1 X + \ldots + a_n X_n \in \mathbb{R}[X]$, initial approximation $x_0 \in \mathbb{R}$.
- For $k = 0, 1, \ldots$ iterate until $a(x_k) \approx 0$:
  1. Approximate a at x_k by the linear polynomial $l_k(X) := a'(x_k) \cdot (X - x_k) + a(x_k)$
  2. Choose x_{k+1} as the root of l_k: $x_{k+1} := x_k - \frac{a(x_k)}{a'(x_k)}$.

(In general, Newton's method will not converge; choice of x_0 matters.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise: Newton's method (*)



$x_2 \approx 1.462, x_3 \approx 1.41502$

- Irrational roots of polynomials can be approximated using either the bisection method or preferably using Newton's method.

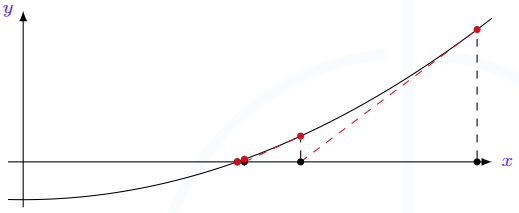
Def Newton's method for polynomials

- Input: $a = a_0 + a_1 X + \ldots + a_n X_n \in \mathbb{R}[X]$, initial approximation $x_0 \in \mathbb{R}$.
- For $k = 0, 1, \ldots$ iterate until $a(x_k) \approx 0$:
  1. Approximate a at x_k by the linear polynomial $l_k(X) := a'(x_k) \cdot (X - x_k) + a(x_k)$
  2. Choose x_{k+1} as the root of l_k: $x_{k+1} := x_k - \frac{a(x_k)}{a'(x_k)}$.

(In general, Newton's method will not converge; choice of x_0 matters.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise: Newton's method (*)



$x_3 \approx 1.415, x_4 \approx 1.41425$

- Irrational roots of polynomials can be approximated using either the bisection method or preferably using Newton's method.

Def Newton's method for polynomials

- Input: $a = a_0 + a_1 X + \ldots + a_n X_n \in \mathbb{R}[X]$, initial approximation $x_0 \in \mathbb{R}$.
- For $k = 0, 1, \ldots$ iterate until $a(x_k) \approx 0$:
  1. Approximate a at x_k by the linear polynomial $l_k(X) := a'(x_k) \cdot (X - x_k) + a(x_k)$
  2. Choose x_{k+1} as the root of l_k: $x_{k+1} := x_k - \frac{a(x_k)}{a'(x_k)}$.

(In general, Newton's method will not converge; choice of x_0 matters.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise: Newton's method (*)

Exc Implement `poly_newton` which takes a polynomial a and an initial approximation x_0 of a root of a and iterates $x_{k+1} = x_k - \frac{a(x_k)}{a'(x_k)}$ as long as defined and $|x_{k+1} - x_k| \geq 10^{-9}$.

Use it again to approximate $\sqrt{2}$. Compare it both to the bisection method and the value of `math.sqrt(2)`.

Exc Use `poly_newton` to approximate the only real root of $X^5 - X + 1$.

Exc In general, a and a' might have common roots. Show that x is a root of $\gcd(a, a')$ of multiplicity m **iff** x is root of a of multiplicity $m + 1$.

Hence $\frac{a(x_k)}{a'(x_k)}$ will be defined as long x_k is sufficiently close to a root of a.

In particular, a and $b := \frac{a}{\gcd(a,a')}$ (after canceling) have the same roots but b has only roots of multiplicity 1.

(For polynomials with integer or rational coefficients, we can compute $\frac{a}{\gcd(a,a')}$ without numerical problems if we use e.g. `Fraction` or avoid division at all.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

2 Polynomials and rational functions
Polynomial functions
Rational functions

For personal use only, no commercial use – TUM CIT Department of Computer Science

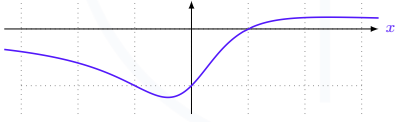
Def A rational expression is built from variables, constants using addition, subtraction, multiplication, and division. It can be always written as a fraction of two polynomials. For $a, b \in \mathbb{R}[X]$ the fraction $\frac{a}{b}$ defines the rational function

$$\mathbb{R} \setminus \{x \in \mathbb{R} \mid b(x) = 0\} \to \mathbb{R}, x \mapsto \frac{a(x)}{b(x)}$$

By completely factorizing a and b and canceling common factors (i.e. their GCD), some "definition gaps" (singularities) can be (continuously) removed.

EG

$$\frac{X^4 - 2X^3 + 2X - 1}{X^5 - X^4 - X + 1} = \frac{(X-1)^3(X+1)}{(X-1)^2(X+1)(X^2+1)} \hat{=} \frac{(X-1)}{(X^2+1)}$$



For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

Exc A polynomial fraction is a pair of polynomials.

Use the so-far implemented functions for polynomials to implement the corresponding functions `pf_add`, `pf_mul`, ... for polynomial fractions (see the code).

Colab might complete most of the code for you (but better check what it proposes).

In particular, implement a function which cancels the numerator and denominator of a polynomial fraction by their GCD.

Exc (*) Implement a function which uses bisection to search for a local extremum of a given rational function $f(x) = \frac{p(x)}{q(x)}$ within a given interval $[x_l, x_r]$:

To this end implement `poly_diff` and `pf_diff` to compute the derivative via $f' = \frac{p'}{q} - f \cdot \frac{q'}{q}$.

Recall, if f has a local extremum at x_0, then necessarily $f'(x_0) = 0$.

To decide if x_0 is indeed a local maximum or minimum, while $f^{(i)}(x_0) = 0$, try to compute $f^{(i+1)}(x_0)$ until either undefined or nonzero. In the later case $f(x)$ behaves locally at x_0 as $f(x_0) + \frac{f^{(i+1)}(x_0)}{(i+1)!} \cdot (x - x_0)^{i+1}$.

Exc Check your implementation using the example given on the previous slide.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises: Classes

- In a nutshell, a number like 1 is an object and an element of the set $\mathbb{Z}$ of integers. But $\mathbb{Z}$ has actually more "structure" as it comes with operations like addition and multiplication (what we have implemented before), i.e. $\mathbb{Z}$ is actually an algebraic structure $(\mathbb{Z}, +, \cdot, 0, 1)$ (called a ring). Similarly, polynomial fractions are an algebraic structure.

  A `class` in a programming language tells the computer how to both store elements of a specific set and also how to compute with them.

  In Python, a `class` can even re-define the meaning of `+,-,*,/`.

Exc Aks Gemini (politely): Please, wrap the functions starting with `pf_` into a class `PolyFrac` which internally stores a pair `p,q` of type `Coeffs` which is meant to represent the polynomial fraction $\frac{p(X)}{q(X)}$ in a single variable X.

The class should support the standard arithmetic operations `+,-,*,/` also if one argument is of type `Number`. Further, it should have the following methods:

- `X` for returning an object which represents the single variable X.
- `cancel` to divide `p,q` by their GCD.
- `diff` for computing the derivative.
- `__call__` to evaluate the polynomial fraction at the given number.

(You can ask Gemini to add some examples and also to add an implementation of `__pow__` for integer exponents afterwards.)

1 Elementary algorithms and arithmetic

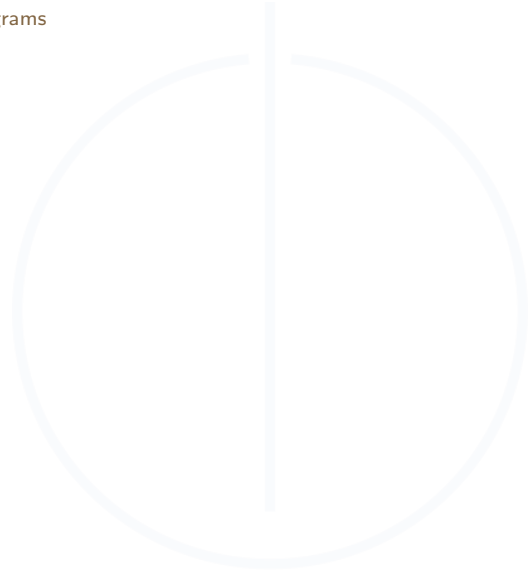
2 Polynomials and rational functions

3 Euclidean geometry in the plane

4 Solutions

For personal use only, no commercial use – TUM CIT Department of Computer Science

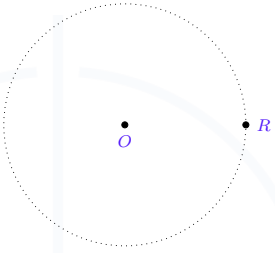
3. Euclidean geometry in the plane
   - Constructive geometry and programs
   - Analytic geometry



For personal use only, no commercial use – TUM CIT Department of Computer Science

Constructing a regular pentagon

```
c = CircleAB(O,R)
g = Line(O,R)
h = PerpendicularLine(O,R,O)
A = Midpoint(O,R)
B = intersection(h,c)[1]
i = AngleBisector(B,A,O)
C = intersection(i,h)[0]
j = PerpendicularLine(C,B,C)
D, E = intersection(c,j)
F = intersection(c,CircleAB(D,B))[1]
G = intersection(c,CircleAB(E,B))[1]
```



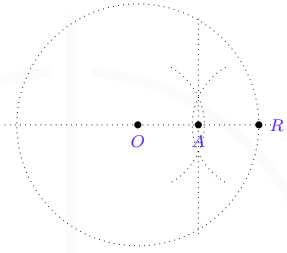
- One of the earliest form of programs (without loops) are straightedge-and-compass constructions (draw and intersect lines and circles).
- One long-standing questions was which regular n-sided polygons can be constructed using straightedge and compass given two initial points O, R.
- Correctness: Is $BEFGD$ indeed a regular pentagon?

  (Either a prove or at least verify that all resulting edges have the same length.)
- Efficiency: Can the pentagon be constructed in less steps?
- Constructability/computability of the general problem: For which $n \in \mathbb{N}$ can we construct regular n-sided polygons using only straightedge and compass?

For personal use only, no commercial use – TUM CIT Department of Computer Science

Constructing a regular pentagon

```
c = CircleAB(O,R)
g = Line(O,R)
h = PerpendicularLine(O,R,O)
A = Midpoint(O,R)
B = intersection(h,c)[1]
i = AngleBisector(B,A,O)
C = intersection(i,h)[0]
j = PerpendicularLine(C,B,C)
D, E = intersection(c,j)
F = intersection(c,CircleAB(D,B))[1]
G = intersection(c,CircleAB(E,B))[1]
```



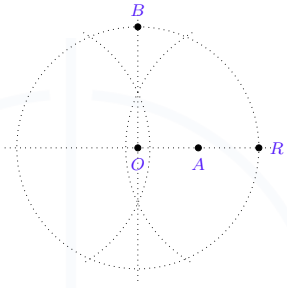
- One of the earliest form of programs (without loops) are straightedge-and-compass constructions (draw and intersect lines and circles).
- One long-standing questions was which regular n-sided polygons can be constructed using straightedge and compass given two initial points O, R.
- Correctness: Is $BEFGD$ indeed a regular pentagon?

  (Either a prove or at least verify that all resulting edges have the same length.)
- Efficiency: Can the pentagon be constructed in less steps?
- Constructability/computability of the general problem: For which $n \in \mathbb{N}$ can we construct regular n-sided polygons using only straightedge and compass?

For personal use only, no commercial use – TUM CIT Department of Computer Science

Constructing a regular pentagon

```
c = CircleAB(O,R)
g = Line(O,R)
h = PerpendicularLine(O,R,O)
A = Midpoint(O,R)
B = intersection(h,c)[1]
i = AngleBisector(B,A,O)
C = intersection(i,h)[0]
j = PerpendicularLine(C,B,C)
D, E = intersection(c,j)
F = intersection(c,CircleAB(D,B))[1]
G = intersection(c,CircleAB(E,B))[1]
```



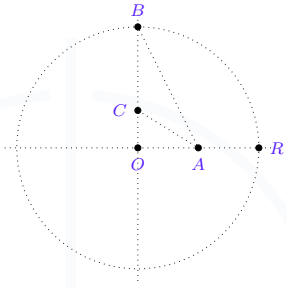
- One of the earliest form of programs (without loops) are straightedge-and-compass constructions (draw and intersect lines and circles).
- One long-standing questions was which regular n-sided polygons can be constructed using straightedge and compass given two initial points O, R.
- Correctness: Is $BEFGD$ indeed a regular pentagon?

  (Either a prove or at least verify that all resulting edges have the same length.)
- Efficiency: Can the pentagon be constructed in less steps?
- Constructability/computability of the general problem: For which $n \in \mathbb{N}$ can we construct regular n-sided polygons using only straightedge and compass?

For personal use only, no commercial use – TUM CIT Department of Computer Science

Constructing a regular pentagon

```
c = CircleAB(O,R)
g = Line(O,R)
h = PerpendicularLine(O,R,O)
A = Midpoint(O,R)
B = intersection(h,c)[1]
i = AngleBisector(B,A,O)
C = intersection(i,h)[0]
j = PerpendicularLine(C,B,C)
D, E = intersection(c,j)
F = intersection(c,CircleAB(D,B))[1]
G = intersection(c,CircleAB(E,B))[1]
```



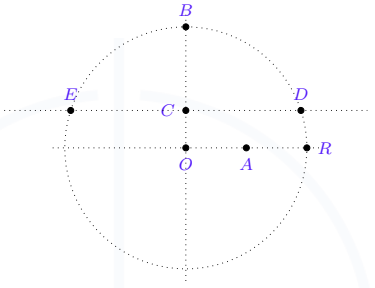
- One of the earliest form of programs (without loops) are straightedge-and-compass constructions (draw and intersect lines and circles).
- One long-standing questions was which regular n-sided polygons can be constructed using straightedge and compass given two initial points O, R.
- Correctness: Is $BEFGD$ indeed a regular pentagon?

  (Either a prove or at least verify that all resulting edges have the same length.)
- Efficiency: Can the pentagon be constructed in less steps?
- Constructability/computability of the general problem: For which $n \in \mathbb{N}$ can we construct regular n-sided polygons using only straightedge and compass?

For personal use only, no commercial use – TUM CIT Department of Computer Science

Constructing a regular pentagon

```
c = CircleAB(O,R)
g = Line(O,R)
h = PerpendicularLine(O,R,O)
A = Midpoint(O,R)
B = intersection(h,c)[1]
i = AngleBisector(B,A,O)
C = intersection(i,h)[0]
j = PerpendicularLine(C,B,C)
D, E = intersection(c,j)
F = intersection(c,CircleAB(D,B))[1]
G = intersection(c,CircleAB(E,B))[1]
```



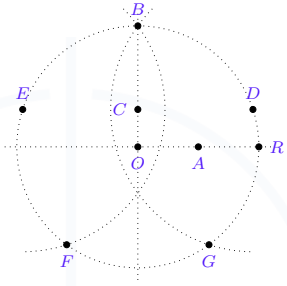
- One of the earliest form of programs (without loops) are straightedge-and-compass constructions (draw and intersect lines and circles).
- One long-standing questions was which regular n-sided polygons can be constructed using straightedge and compass given two initial points O, R.
- Correctness: Is $BEFGD$ indeed a regular pentagon?

  (Either a prove or at least verify that all resulting edges have the same length.)
- Efficiency: Can the pentagon be constructed in less steps?
- Constructability/computability of the general problem: For which $n \in \mathbb{N}$ can we construct regular n-sided polygons using only straightedge and compass?

For personal use only, no commercial use – TUM CIT Department of Computer Science

Constructing a regular pentagon

```
c = CircleAB(O,R)
g = Line(O,R)
h = PerpendicularLine(O,R,O)
A = Midpoint(O,R)
B = intersection(h,c)[1]
i = AngleBisector(B,A,O)
C = intersection(i,h)[0]
j = PerpendicularLine(C,B,C)
D, E = intersection(c,j)
F = intersection(c,CircleAB(D,B))[1]
G = intersection(c,CircleAB(E,B))[1]
```



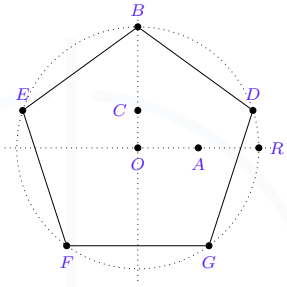
- One of the earliest form of programs (without loops) are straightedge-and-compass constructions (draw and intersect lines and circles).
- One long-standing questions was which regular n-sided polygons can be constructed using straightedge and compass given two initial points O, R.
- Correctness: Is $BEFGD$ indeed a regular pentagon?

  (Either a prove or at least verify that all resulting edges have the same length.)
- Efficiency: Can the pentagon be constructed in less steps?
- Constructability/computability of the general problem: For which $n \in \mathbb{N}$ can we construct regular n-sided polygons using only straightedge and compass?

For personal use only, no commercial use – TUM CIT Department of Computer Science

Constructing a regular pentagon

```
c = CircleAB(O,R)
g = Line(O,R)
h = PerpendicularLine(O,R,O)
A = Midpoint(O,R)
B = intersection(h,c)[1]
i = AngleBisector(B,A,O)
C = intersection(i,h)[0]
j = PerpendicularLine(C,B,C)
D, E = intersection(c,j)
F = intersection(c,CircleAB(D,B))[1]
G = intersection(c,CircleAB(E,B))[1]
```



- One of the earliest form of programs (without loops) are straightedge-and-compass constructions (draw and intersect lines and circles).
- One long-standing questions was which regular n-sided polygons can be constructed using straightedge and compass given two initial points O, R.
- Correctness: Is $BEFGD$ indeed a regular pentagon?

  (Either a prove or at least verify that all resulting edges have the same length.)
- Efficiency: Can the pentagon be constructed in less steps?
- Constructability/computability of the general problem: For which $n \in \mathbb{N}$ can we construct regular n-sided polygons using only straightedge and compass?

For personal use only, no commercial use – TUM CIT Department of Computer Science



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

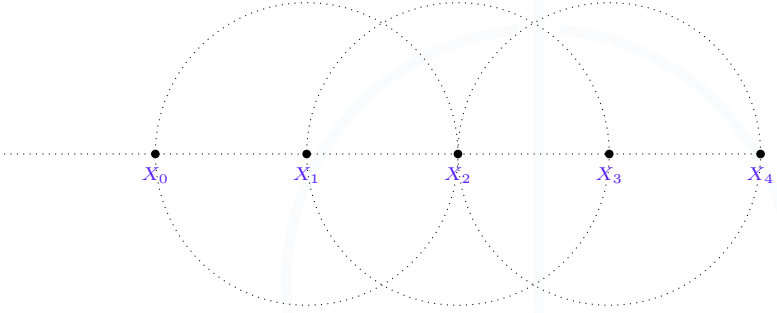
For personal use only, no commercial use – TUM CIT Department of Computer Science



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

For personal use only, no commercial use – TUM CIT Department of Computer Science

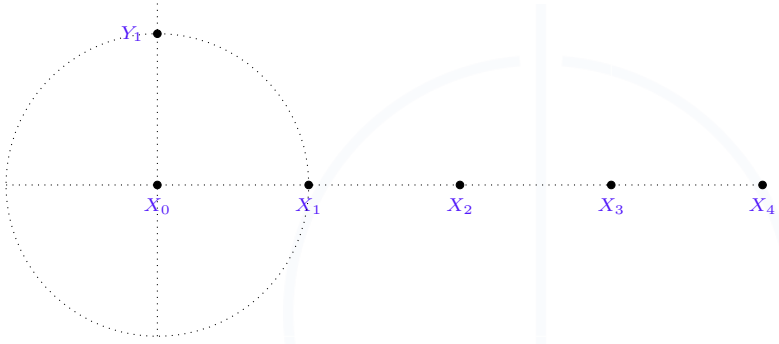
Straightedge and compass as calculator for rational numbers



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

For personal use only, no commercial use – TUM CIT Department of Computer Science

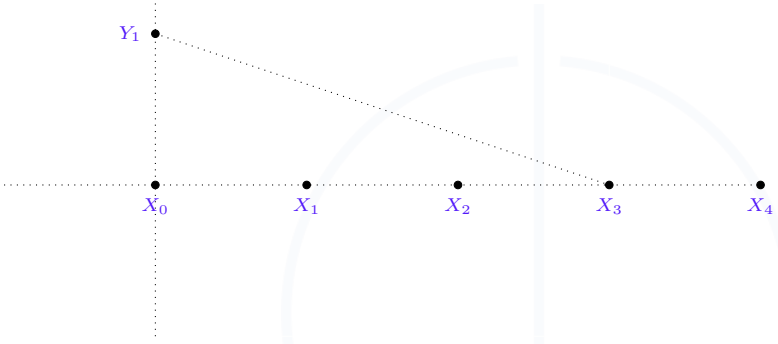
Straightedge and compass as calculator for rational numbers



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

For personal use only, no commercial use – TUM CIT Department of Computer Science

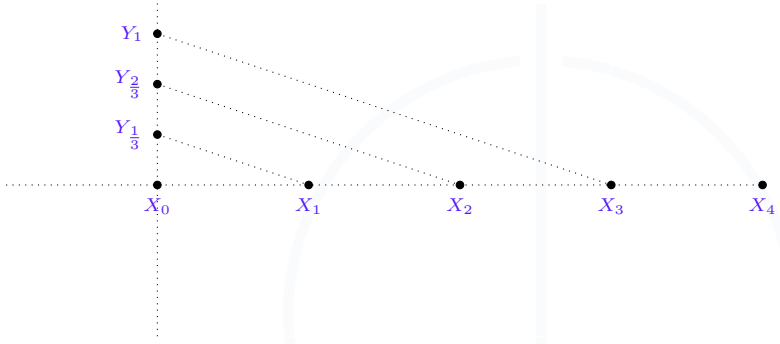
Straightedge and compass as calculator for rational numbers



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

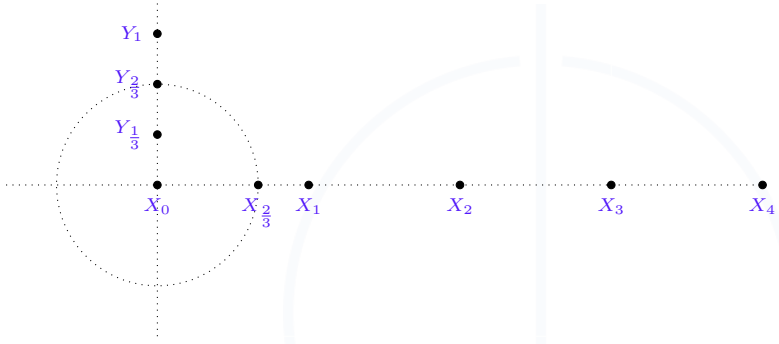
For personal use only, no commercial use – TUM CIT Department of Computer Science

Straightedge and compass as calculator for rational numbers



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

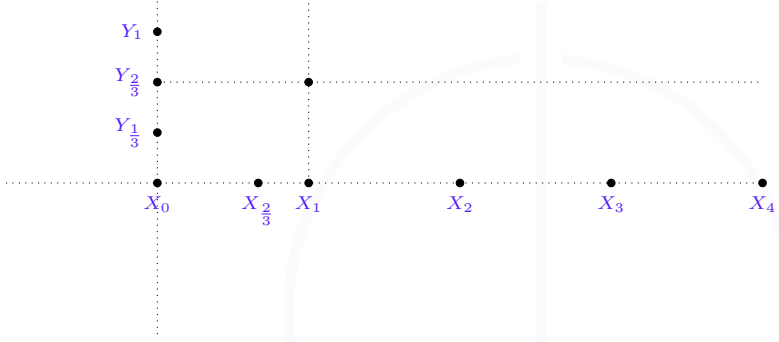
For personal use only, no commercial use – TUM CIT Department of Computer Science



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

For personal use only, no commercial use – TUM CIT Department of Computer Science

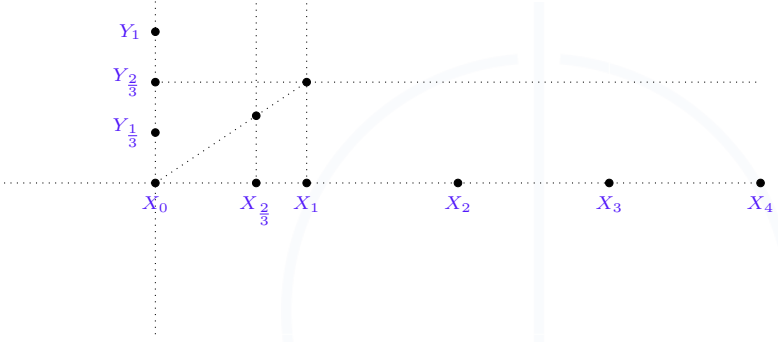
Straightedge and compass as calculator for rational numbers



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

For personal use only, no commercial use – TUM CIT Department of Computer Science

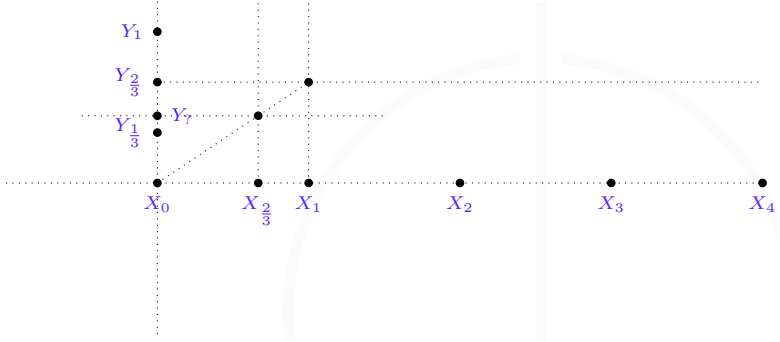
Straightedge and compass as calculator for rational numbers



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.

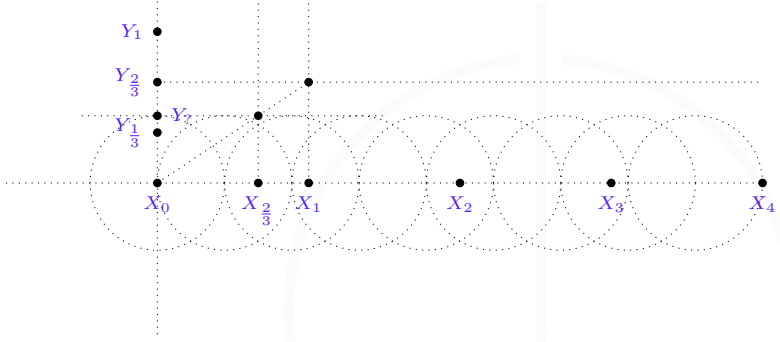
For personal use only, no commercial use – TUM CIT Department of Computer Science

Straightedge and compass as calculator for rational numbers



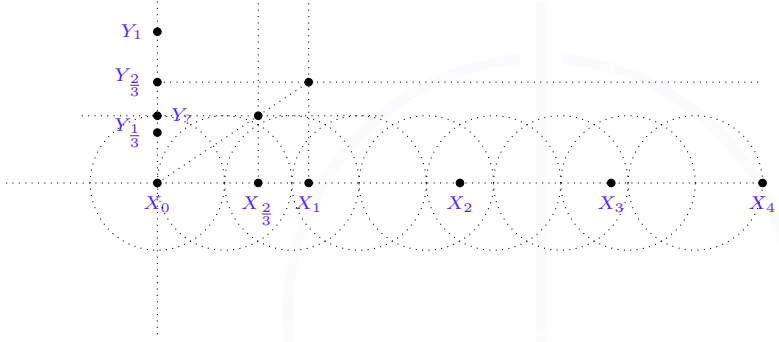
- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.
- By the intercept theorem, $Y_?$ should have distance $\frac{2}{3} \cdot \frac{2}{3}$ from X_0.
- We can check that $9 \cdot (\frac{2}{9} \cdot \frac{2}{9}) = 4$ using the compass.

For personal use only, no commercial use – TUM CIT Department of Computer Science



- Two distinct points X_0, X_1 define a "number line" and thus $\mathbb{Z}$.
- Straightedge and compass allow to construct bisectors, perpendicular lines, parallels, and thus rational numbers by virtue of the intercept theorem.
- By the intercept theorem, $Y_?$ should have distance $\frac{2}{3} \cdot \frac{2}{3}$ from X_0.
- We can check that $9 \cdot (\frac{2}{9} \cdot \frac{2}{9}) = 4$ using the compass.

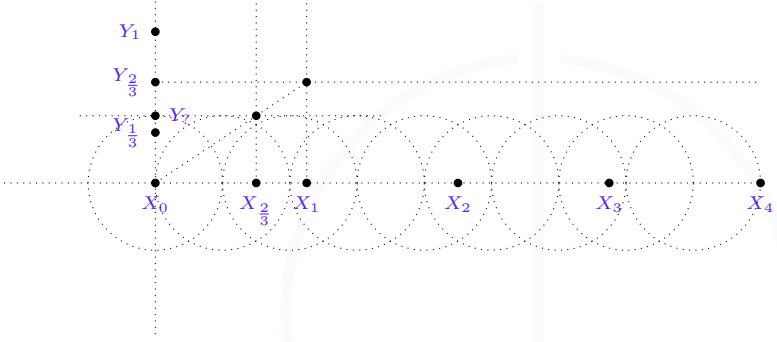
For personal use only, no commercial use – TUM CIT Department of Computer Science



- Going by Peter Shor, Euclid called certain numbers (lengths) "alogos" ("unsayable", "unreasonable") which in Latin became "irrationalis".
- "Rational" numbers are "explainable by means of a constructive finite and loop-free program".
- But also e.g. $\sqrt{1 + (2/3)^2}$ is "explainable/constructible".

  (In fact, every square root $\sqrt{z}$ for $z \in \mathbb{Q}_{\geq 0}$ is constructible; cf. geometric mean theorem).

For personal use only, no commercial use – TUM CIT Department of Computer Science



- ⚠ A construction in the strict sense is a (finite) program without loops based on the elementary constructions of drawing and intersecting lines and circles.
- ▷ But not every real number can be constructed in this way starting with X_0, X_1:
  Essentially, we can only intersect lines and circles with each other which means that a constructible number can be written as an arithmetic expression using the standard operations $+, -, \cdot, /$ and further $\sqrt[2]{\cdot}$ (SymPy does this internally).

For personal use only, no commercial use – TUM CIT Department of Computer Science

What is constructible? What is computable?*

- ⚠ The available actions (rules, constructions, instructions, commands, ...) can impose limits to that what can be constructed or computed.
- ▷ E.g. given X_0, X_1, straightedge and compass only allow to solve linear and quadratic equations over $\mathbb{Q}$:
  - ▷ The Gauss–Wantzel theorem that a regular n-gon is constructible if and only if $n = 2^e \cdot p_1 \cdots p_r$ with $p_1 < \ldots < p_r$ distinct Fermat primes:

    A Fermat prime is of the form $2^{2^k} - 1$; currently we only know the Fermat primes $3, 5, 17, 257, 65537$ ($0 \leq k \leq 4$), but e.g. 641 divides $2^{2^5} + 1$.
  - ▷ E.g. given X_0, X_1, we can construct a regular n-gon for $n = 3, 4, 5, 6, 8, 10$ but not for $n = 7, 9$.
  - ▷ In particular, this means that we can trisect $180°$ to obtain $60°$, but it must be impossible to trisect $60°$ further to obtain $20°$.
  - ▷ Using $(\cos 3\theta) = 4(\cos\theta)^3 - 3(\cos\theta)$, this means that we cannot solve cubic equations in general over $\mathbb{Q}$.
  - ▷ More impossible constructions: squaring the circle, doubling the cube.

    (But e.g. if we allow "angle trisection" as additional instruction, then we can also construct e.g. a regular 9-gon (nona-/ennea-gon).)

For personal use only, no commercial use – TUM CIT Department of Computer Science

What is constructible? What is computable?*

- ⚠ The available actions (rules, constructions, instructions, commands, ...) can impose limits to that what can be constructed or computed.
- ▷ Similarly, we can say that a real number $x_0 \in \mathbb{R}$ is "computable" if there is a program that on input i, b computes the first i digits of x_0 w.r.t. base b where the program is allowed to use standard arithmetic operations (as we have implemented) and loops.
- ▷ Then $\sqrt[3]{x_0}$ is computable for every $x_0 \in \mathbb{Q}_{\geq 0}$ (and by extension also if $x_0 \in \mathbb{R}$ is computable) using bisection (or preferably Newton's method) to approximate the root of $x^3 - x_0$.
- ▷ But there also exist irrational numbers $x \in \mathbb{R} \setminus \mathbb{Q}$ which cannot be computed.

  (At least based the Church-Turing thesis.)
- ▷ In analytic geometry, we are not restricted to straightedge and compass, and can work with arbitrary real numbers.

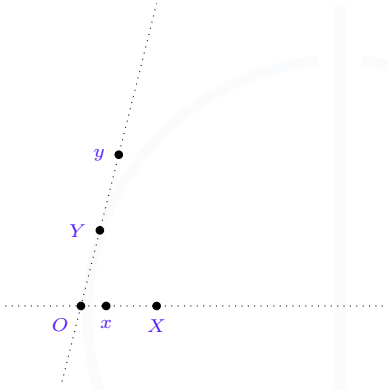
For personal use only, no commercial use – TUM CIT Department of Computer Science

3 Euclidean geometry in the plane

Constructive geometry and programs

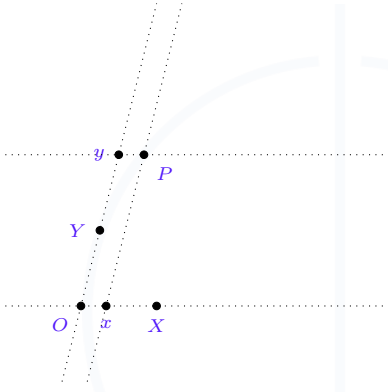
Analytic geometry

For personal use only, no commercial use – TUM CIT Department of Computer Science



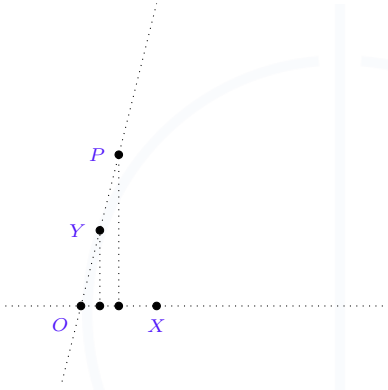
- Given three points in "general position" we map every pair $(x, y) \in \mathbb{Q}^2$ of rational numbers to a point:
  - First construct x and y and the respective lines OX and OY with $|OX|$ and $|OY|$ the corresponding unit of measure.
  - Then let (x, y) define the point P in which the parallel through x and y intersect.
  - (x, y) are the coordinates for P w.r.t. O, X, Y.

For personal use only, no commercial use – TUM CIT Department of Computer Science



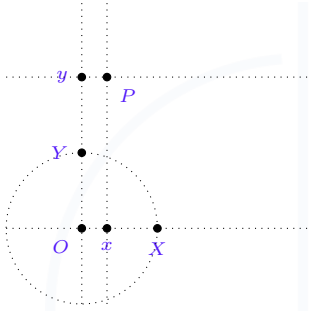
- Given three points in "general position" we map every pair $(x, y) \in \mathbb{Q}^2$ of rational numbers to a point:
  - First construct x and y and the respective lines OX and OY with $|OX|$ and $|OY|$ the corresponding unit of measure.
  - Then let (x, y) define the point P in which the parallel through x and y intersect.
  - (x, y) are the coordinates for P w.r.t. O, X, Y.

For personal use only, no commercial use – TUM CIT Department of Computer Science



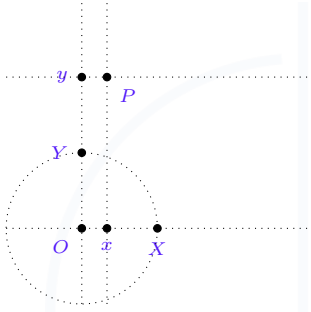
- If the lines OX and OY are not orthogonal/perpendicular, when "moving a point P" e.g. along OY the orthogonal projection of P onto OX also changes:
  - The orthogonal projection of P onto OX is obtained by constructing the line perpendicular to OX through P and then intersecting this line with OX.
- But if OX and OY are orthogonal ($OX \perp OY$), then for every P on OY the line perpendicular to OX through P is OY, i.e. the projection is always O.

For personal use only, no commercial use – TUM CIT Department of Computer Science



- Given two distinct points O, X the Cartesian interpretation of a pair $(x, y) \in \mathbb{Q}^2$ (or $(x, y) \in \mathbb{R}^2$) as coordinates of a point in the plane is:
  - take OX as number line and fix Y as one intersection of the line perpendicular to OX through O and the circle with center O through X (OX: x-axis; OY: y-axis);
  - construct x, y on the corresponding lines and let (x, y) denote the point P in which the parallels through x and y intersect.

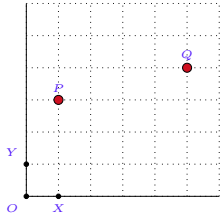
For personal use only, no commercial use – TUM CIT Department of Computer Science

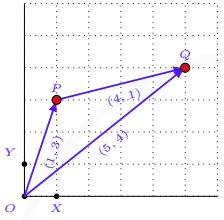


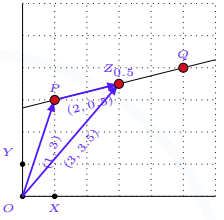
- This way we use the same unit of measure $|OX| = |OY|$ on both lines and the coordinates are independent of each other, and the Cartesian coordinates of O, X, Y are $(0,0)$, $(1,0)$ and $(0,1)$, respectively.
- Recall that by the very definition of $\mathbb{R}$, every $x \in \mathbb{R}$ is the limit of a sequence of rational numbers, i.e. for every $\varepsilon > 0$ there is some $\frac{p}{q} \in \mathbb{Q}$ s.t. $\left|x - \frac{p}{q}\right| < \varepsilon$.
- I.e. also every $(x, y) \in \mathbb{R}^2$ defines a unique point in the plane in this way (although the point might be neither constructible nor computable).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Coordinate vectors: position vs. displacement



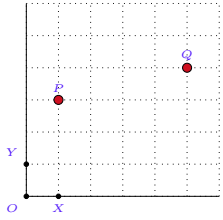


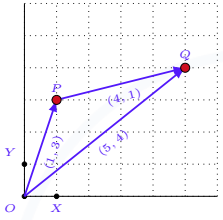


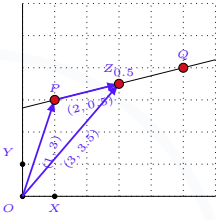
- We assume in the following that O, X, Y are fixed with O the origin and $1 = |OX| = |OY|$ and $OX \perp OY$.
  - For a point P write $(P_x, P_y) \in \mathbb{R}^2$ for its coordinates w.r.t. O, X, Y.
    (We could also use (x_P, y_P); there is not standard notation.)
  - For two points P, Q let $Q - P := (Q_x - P_x, Q_y - P_y)$ denote the "displacement of Q from P".Thus, $(P_x, P_y) = P - O$ can also be understood as the displacement of P from the origin O.
  - A pair $(x, y) \in \mathbb{R}^2$ is thus also called "(coordinate) vector" ("carrier", "bearer", "transporter").

For personal use only, no commercial use – TUM CIT Department of Computer Science

Coordinate vectors: position vs. displacement



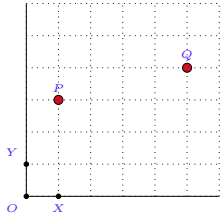


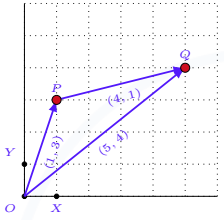


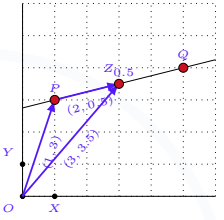
- There is no standard notation for (coordinate) vectors: typically, lower-case letters are used, e.g. just v or for emphasis $\boldsymbol{v}$ or $\underline{v}$ or $\vec{v}$.
- We will use $\boldsymbol{v} = (v_x, v_y)$ in the following and write $P = \boldsymbol{p}$ to denote a point with coordinates $\boldsymbol{p}$.
- Addition $\boldsymbol{v} + \boldsymbol{w} := (v_x + w_x, v_y + w_y)$ of two (coordinate) vectors $\boldsymbol{v}, \boldsymbol{w}$ is defined as the result of "first displace by $\boldsymbol{v}$; then displace by $\boldsymbol{w}$".
  - Obviously, we have $\boldsymbol{v} + \boldsymbol{w} = \boldsymbol{w} + \boldsymbol{v}$ ("parallelogram").
- We can scale a vector $\boldsymbol{v}$ by multiplying it with a "scalar $\lambda \in \mathbb{R}$" producing the vector $\lambda\boldsymbol{v} = (\lambda v_x, \lambda v_y)$.
  - In particular $-\boldsymbol{v}$ means to reverse the direction, so $\boldsymbol{v} + (-\boldsymbol{v}) = (0, 0)$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Coordinate vectors: position vs. displacement







- Given two points $P = \boldsymbol{p}, Q = \boldsymbol{q}$, the line PQ is the set of points

$$\{\boldsymbol{p} + \lambda(\boldsymbol{q} - \boldsymbol{p}) \in \mathbb{R}^2 \mid \lambda \in \mathbb{R}\}$$

  - We can also consider PQ as number line with P as origin: on this line, we can define for every scalar $\lambda \in \mathbb{R}$ a point Z_λ s.t. $Z_0 = P$ and $Z_1 = Q$ (and also every point on PQ defines some λ as in the case of OX).
  - Then Z_λ must have the coordinates $\boldsymbol{p} + \lambda \cdot (\boldsymbol{q} - \boldsymbol{p})$.
  - Similarly the line segment $\overline{PQ}$ is the set $\{\boldsymbol{p} + \lambda(\boldsymbol{q} - \boldsymbol{p}) \mid \lambda \in [0, 1]\}$ and the ray $\vec{PQ}$ is the set $\{\boldsymbol{p} + \lambda(\boldsymbol{q} - \boldsymbol{p}) \mid \lambda \in \mathbb{R}_{\geq 0}\}$ (always w.r.t. the chosen points of reference O, X, Y).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercise

```
from typing import *
import math

type Scalar = float

class Vc(NamedTuple):
  x: Scalar
  y: Scalar

  def __add__(self, other: 'Vc') -> 'Vc':       # v + w = (v_x + w_x, v_y + w_y)
    return Vc(self.x+other.x, self.y+other.y)

  def __mul__(self, other: Scalar) -> 'Vc':     # v · λ = (v_x · λ, v_y · λ)
    return Vc(self.x*other, self.y*other)
```

- ▷ Formally, a coordinate vector is a finite sequence, a tuple. In the plane, we only need two coordinates. In Python we can use `NamedTuple` to give the entries the names `x,y`.
- ▷ Python allows to overwrite the definition of the symbols `+,*,-,/`. Above code already implements vector addition and scalar multiplication (note: currently, the scalar must come from the right).

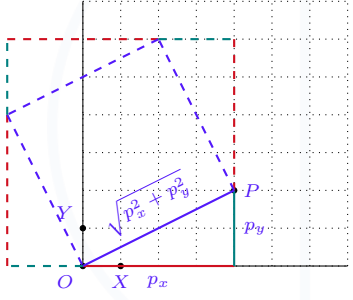
Exc Overwrite `__sub__`, `__truediv__` and `__rmul__` to implement $\boldsymbol{v} - \boldsymbol{w}$, $\boldsymbol{v}/\lambda$ and $\lambda \cdot \boldsymbol{v}$ (we ignore rounding problems).

For personal use only, no commercial use – TUM CIT Department of Computer Science

- Given O, X and some point $P = \boldsymbol{p}$, the distance $|OP|$ is the length of $\boldsymbol{p}$

$$|OP| = \sqrt{p_x^2 + p_y^2} =: \|\boldsymbol{p}\|_2$$

as $|OP|^2 = (p_x + p_y)^2 - 4 \cdot \frac{1}{2} \cdot p_x \cdot p_y = p_x^2 + p_y^2$ by the Pythagorean theorem:



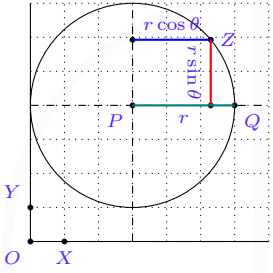
- Analogously, the distance of two points P and Q is

$$|PQ| = \|\boldsymbol{q} - \boldsymbol{p}\|_2 = \sqrt{(q_x - p_x)^2 + (q_y - p_y)^2}$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Circle in the Euclidean plane

- The circle with center P through Q consists of the points Z with $|PZ| = |PQ|$.



- If $P = \boldsymbol{p}$, $Q = \boldsymbol{q}$ and $Z = \boldsymbol{z}$, then the circle consists of the points (w.r.t. O, X, Y)

$$\{\boldsymbol{z} \in \mathbb{R}^2 \mid \|\boldsymbol{z} - \boldsymbol{p}\|_2^2 = \|\boldsymbol{q} - \boldsymbol{p}\|_2^2\}$$

or by definition of $\sin$ and $\cos$

$$\{\boldsymbol{p} + \|\boldsymbol{q} - \boldsymbol{p}\|_2 \cdot (\cos(\theta), \sin(\theta)) \in \mathbb{R}^2 \mid \theta \in \mathbb{R}\}$$

(Recall: $\sin, \cos$ are defined to be the orthogonal projections of a point on the unit circle onto the two axes.)

For personal use only – no commercial use – TUM CIT Department of Computer Science

Exercises: drawing using `matplotlib`

- We cannot draw the (uncountably many) points a line or a circle consists of.
- Instead, we draw sufficiently many points as required to give the impression of a closed line or circle, i.e. we fix a sufficiently large $n \in \mathbb{N}$ and draw e.g.

$$\{\boldsymbol{p} + \lambda(\boldsymbol{q} - \boldsymbol{p}) \in \mathbb{R}^2 \mid \lambda \in \{0, \tfrac{1}{n}, \tfrac{2}{n}, \ldots, \tfrac{n}{n}\}\}$$

  respectively

$$\{\boldsymbol{p} + \|\boldsymbol{q} - \boldsymbol{p}\|_2 \cdot (\cos(\theta), \sin(\theta)) \in \mathbb{R}^2 \mid \theta \in \{0, \tfrac{2\pi}{n}, \tfrac{4\pi}{n}, \tfrac{6\pi}{n}, \ldots, \tfrac{2(n-1)\pi}{n}\}\}$$

  i.e. we "discretize" the line and circle (from $\mathbb{R}$ back to $\mathbb{Q}$).
- The provided code already contains methods `plot` for drawing a `Vc` (as a cross for better visibility) and a `Ln` (`matplotlib` does the discretization for us).

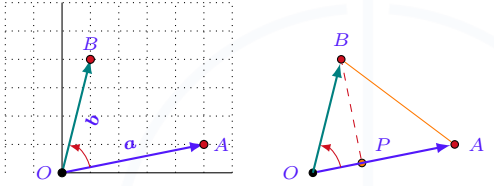
Exc Use `matplotlib` to draw a circle:

- First extend `Vc` by methods `snorm` and `norm` which return $\|\boldsymbol{v}\|_2^2$ and $\|\boldsymbol{v}\|_2$.
- Implement also a method `plot` to draw a `Cr`: approximate the circle by a regular n-gon; to draw several points $(x_1, y_1), \ldots, (x_k, y_k)$ and connect them using line segments, you can use `plot([x_1,...,x_k],[y_1,...,y_k],'-')`.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Euclidean scalar product in the plane

- Let $\alpha = \angle AOB$, $A = \boldsymbol{a}$, $B = \boldsymbol{b}$.



- From the Pythagorean theorem the law of cosine follows (here assuming $\alpha \leq \frac{\pi}{2}$):
  - $\cos\alpha = \frac{|OP|}{|OB|}$, $\sin\alpha = \frac{|PB|}{|OB|}$ (definition of $\cos, \sin$)
  - $|AB|^2 = |PB|^2 + |PA|^2 = |OB|^2 + |OA|^2 - 2|OA||OB|\cos\alpha$

$$\cos\alpha = \frac{|OA|^2+|OB|^2-|AB|^2}{2|OA||OB|} = \frac{\|\boldsymbol{a}\|_2^2+\|\boldsymbol{b}\|_2^2-\|\boldsymbol{a}-\boldsymbol{b}\|_2^2}{2\|\boldsymbol{a}\|_2\|\boldsymbol{b}\|_2}$$

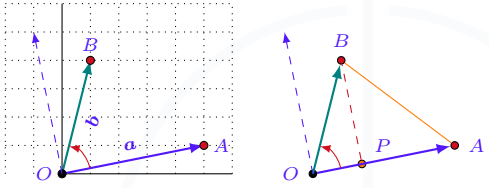
As $\|\boldsymbol{a}\|_2^2 = a_x^2 + b_y^2$:

$$\|\boldsymbol{a}-\boldsymbol{b}\|_2^2 = \|\boldsymbol{a}\|_2^2 + \|\boldsymbol{b}\|_2^2 - 2\cdot(a_x\cdot b_x + a_y\cdot b_y)$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Euclidean scalar product in the plane

- Let $\alpha = \angle AOB$, $A = \boldsymbol{a}$, $B = \boldsymbol{b}$.



- $\langle \boldsymbol{a}, \boldsymbol{b} \rangle_2 := a_x \cdot b_x + a_y \cdot b_y$ is the Euclidean scalar product (product is a scalar).
- ⚠ $\|\boldsymbol{a}\|_2^2 = \langle \boldsymbol{a}, \boldsymbol{a} \rangle_2$ and $\|\lambda \boldsymbol{a} + \mu \boldsymbol{b}\|_2^2 = \lambda^2 \|\boldsymbol{a}\|_2^2 + 2\lambda\mu \langle \boldsymbol{a}, \boldsymbol{b} \rangle_2 + \mu^2 \|\boldsymbol{b}\|_2^2$.
- We thus have for the angle $\alpha = \angle AOB$:

$$\cos \alpha = \frac{\langle \boldsymbol{a}, \boldsymbol{b} \rangle_2}{\|\boldsymbol{a}\|_2 \|\boldsymbol{b}\|_2} \in [-1, 1]$$

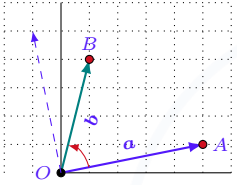
The (nonzero) vectors $\boldsymbol{a}, \boldsymbol{b}$ are orthogonal ($\boldsymbol{a} \perp \boldsymbol{b}$) **iff** $\langle \boldsymbol{a}, \boldsymbol{b} \rangle_2 = 0$.

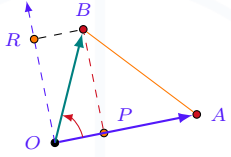
- In the plane we always have $\langle (x, y), (-y, x) \rangle_2 = 0$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Euclidean scalar product in the plane

- Let $\alpha = \angle AOB$, $A = \boldsymbol{a}$, $B = \boldsymbol{b}$.





- The orthogonal projection P of B onto OA has the coordinates:

$$\|\boldsymbol{b}\|_2 \cos\alpha \cdot \frac{\boldsymbol{a}}{\|\boldsymbol{a}\|_2} = \frac{\langle \boldsymbol{a}, \boldsymbol{b} \rangle_2}{\|\boldsymbol{a}\|_2^2} \cdot \boldsymbol{a}$$

(If $\|a\|_2 = 1$, then $\langle \boldsymbol{a}, \boldsymbol{b} \rangle_2$ is the scalar by which we scale $\boldsymbol{a}$ to obtain the coordinates of P.)

- Thus if $\boldsymbol{n}$ is orthogonal to $\boldsymbol{a}$ (replace $\boldsymbol{a}$ by $\boldsymbol{n}$):

$$|PB| = |OR| = \| \frac{\langle \boldsymbol{n}, \boldsymbol{b} \rangle_2}{\|\boldsymbol{n}\|_2^2} \cdot \boldsymbol{n} \|_2 = \frac{\left| \langle \boldsymbol{n}, \boldsymbol{b} \rangle_2 \right|}{\|\boldsymbol{n}\|_2}$$

and B is a point of OA **iff** $|PB| = 0$ **iff** $\langle \boldsymbol{n}, \boldsymbol{b} \rangle_2 = 0$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Intersection of two lines in the plane

- Given: lines AB and CD.
- Assume coordinates $A = \boldsymbol{a}$, $B = \boldsymbol{b}$, $C = \boldsymbol{c}$ and $D = \boldsymbol{d}$.
- Let $\boldsymbol{n} \perp (\boldsymbol{b} - \boldsymbol{a})$ and $\delta := \langle n, \boldsymbol{a} \rangle_2$. Then

$$\{\boldsymbol{z} \in \mathbb{R}^2 \mid \delta = \langle n, \boldsymbol{z} \rangle_2\}$$

is the Hesse normal form of AB.

(δ is the oriented distance of the origin to AB measured w.r.t. $\|\boldsymbol{n}\|_2$.)

The parametric form of CD is:

$$\{\boldsymbol{c} + \lambda(\boldsymbol{d} - \boldsymbol{c}) \mid \lambda \in \mathbb{R}\}$$

Finally, solve:

$$\delta = \langle \boldsymbol{n}, \boldsymbol{c} + \lambda(\boldsymbol{d} - \boldsymbol{c}) \rangle_2$$

i.e.

$$\langle \boldsymbol{n}, (\boldsymbol{d} - \boldsymbol{c}) \rangle_2 \cdot \lambda + \langle \boldsymbol{n}, \boldsymbol{c} \rangle_2 - \delta = 0$$

(There is no solution iff $\langle \boldsymbol{n}, \boldsymbol{d} - \boldsymbol{c} \rangle_2 = 0$ and $\langle \boldsymbol{n}, \boldsymbol{c} \rangle_2 \neq \delta$.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Intersection of a line and circle in the plane

- Given: line AB and circle with center C and D on its perimeter.
- Assume coordinates $A = \boldsymbol{a}$, $B = \boldsymbol{b}$, $C = \boldsymbol{c}$ and $D = \boldsymbol{d}$.
- The parametric form of AB is:

$$\{\boldsymbol{a} + \lambda(\boldsymbol{b} - \boldsymbol{a}) \mid \lambda \in \mathbb{R}\}$$

Let $\rho := \|\boldsymbol{d} - \boldsymbol{c}\|_2$. Then the circle is

$$\{\boldsymbol{z} \in \mathbb{R}^2 \mid \|\boldsymbol{z} - \boldsymbol{c}\|_2 = \rho\}$$

Thus solve

$$\|\boldsymbol{a} + \lambda(\boldsymbol{b} - \boldsymbol{a}) - \boldsymbol{c}\|_2^2 = \rho^2$$

i.e. solve

$$\lambda^2\|\boldsymbol{b} - \boldsymbol{a}\|_2^2 - 2\lambda\langle\boldsymbol{b} - \boldsymbol{a}, \boldsymbol{c} - \boldsymbol{a}\rangle_2 + \|\boldsymbol{c} - \boldsymbol{a}\|_2^2 - \rho^2 = 0$$

(Alternative: project C onto AB to obtain P with coordinates $\boldsymbol{p} := \boldsymbol{a} + \frac{\langle\boldsymbol{b}-\boldsymbol{a}, \boldsymbol{c}-\boldsymbol{a}\rangle_2}{\|\boldsymbol{b}-\boldsymbol{a}\|_2^2} \cdot (\boldsymbol{b} - \boldsymbol{a})$; if $\|\boldsymbol{p} - \boldsymbol{c}\|_2 > \rho$, there is no solution; else the intersections must have distance $\rho^2 - \|\boldsymbol{p} - \boldsymbol{c}\|_2^2$ from P.)

For personal use only, no commercial use · TUM CIT Department of Computer Science

Intersection of two circles in the plane

- Given: circles defined by A, B and C, D (center, point on perimeter)
- Assume coordinates $A = \boldsymbol{a}$, $B = \boldsymbol{b}$, $C = \boldsymbol{c}$ and $D = \boldsymbol{d}$.
- Set $\rho_1 := \|\boldsymbol{b} - \boldsymbol{a}\|_2$, $\rho_2 := \|\boldsymbol{d} - \boldsymbol{c}\|_2$, $\boldsymbol{v} := \boldsymbol{c} - \boldsymbol{a}$ and $\delta := \|\boldsymbol{v}\|_2$.
  Assume the circles intersect in $Q = \boldsymbol{q}$ (i.e. $(\rho_1 - \rho_2)^2 \leq \delta^2 \leq (\rho_1 + \rho_2)^2$).
  Its projection onto the line AC can be written as $\boldsymbol{p} = \boldsymbol{a} + \lambda \boldsymbol{v}$.
  By virtue of Pythagoras we must have

$$\|\boldsymbol{q} - \boldsymbol{p}\|_2^2 = \rho_1^2 - \|\boldsymbol{p} - \boldsymbol{a}\|_2^2 = \rho_2^2 - \|\boldsymbol{p} - \boldsymbol{c}\|_2^2$$

  i.e.

$$0 = \rho_1^2 - \rho_2^2 - \lambda^2 \|\boldsymbol{v}\|_2^2 + (\lambda - 1)^2 \|\boldsymbol{v}\|_2^2 = \rho_1^2 - \rho_2^2 + \|\boldsymbol{v}\|_2^2 - 2\lambda \|\boldsymbol{v}\|_2^2$$

  Thus, if $\delta^2 = \|v\|_2^2 \neq 0$, we have $\lambda = \frac{\rho_1^2 - \rho_2^2}{2\delta^2} + \frac{1}{2}$ with $\boldsymbol{p} = \frac{\boldsymbol{a}+\boldsymbol{c}}{2} + \frac{\rho_1^2 - \rho_2^2}{2\delta^2} \boldsymbol{v}$.
  We have $\|\boldsymbol{p} - \boldsymbol{a}\|_2^2 = \lambda^2 \delta^2$. Assume $\|\boldsymbol{p} - \boldsymbol{a}\|_2^2 = \lambda^2 \delta^2 \leq \rho_1^2 \leq \rho_2^2$.
  The vector $\boldsymbol{q} - \boldsymbol{p}$ is orthogonal to $\boldsymbol{v}$ and thus collinear to $\boldsymbol{w} := (-v_y, v_x)$ s.t.
  $\|\boldsymbol{q} - \boldsymbol{p}\|_2^2 = \mu^2 \cdot \|\boldsymbol{w}\|_2^2$ and thus $\mu^2 = \frac{\rho_1^2 - \lambda^2 \delta^2}{\delta^2}$ (as $\|\boldsymbol{v}\|_2 = \|\boldsymbol{w}\|_2$).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises

Exc Extend `Vc` by methods `__matmul__` and `perp` which return $\langle \boldsymbol{v}, \boldsymbol{w} \rangle_2$ and $(-v_y, v_x)$, respectively.

(`__matmul__` is Python's internal name for `@` and used for matrix multiplication; we use it here for the scalar product (slight abuse of notation).)

Exc Implement functions which compute the sets of all intersections of two lines, two circles, a line with a circle. (Return a list; fix some order.)

Exc Use your implementation to implement the construction of the pentagon from the beginning.

Exc (*) Currently, a line AB is only drawn as the line segment $\overline{AB}$.

Adapt the code so that the line AB spans the whole drawing area (given by its lower-left corner L and its upper-right corner R).

This requires to pass the corners of the drawing area as additional parameters to the `plot` methods.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Exercises: Extension to $\mathbb{R}^3$

Exc (*) In the provided code, you can find also the analogous implementation for three dimensions.

Recall how to intersect lines, planes, spheres with each other in Euclidean space and extend the code accordingly.

- In practice, you would not implement coordinate vectors yourself, but use e.g. NumPy which is more efficient and flexible.

The notation slightly changes, here is a short example.

```
import numpy as np

a = np.array([1,2,3])
b = np.array([3,4,5])
print("Vector addition:", a+a, a+b, b+a, b+b)
print("Scalar multiplication:", 2*a, -a, 3*b, b/2)
print("Euclidean scalar product:", a @ b, a.dot(b), np.dot(a,b))
print("Euclidean norm:", np.linalg.norm(a), np.sqrt(a@a))
```

You can also try to use NumPy instead of the provided implementation for three dimensional coordinate vectors.

For personal use only, no commercial use – TUM CIT Department of Computer Science

1. Elementary algorithms and arithmetic
2. Polynomials and rational functions
3. Euclidean geometry in the plane
4. Solutions

For personal use only, no commercial use – TUM CIT Department of Computer Science

Solutions

- As there are different ways of saying the same thing, there are different ways to tell the computer what it should do.
- The main goal is to keep the description ("code") simple and reuse as much as possible, which means that it is rather inefficient and also some special cases are ignored (potentially leading to uncaught exceptions).

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```python
# THESE ARE LINE COMMENTS, Python ignores this, but the "AI" might read this
# In computer science we also use letters as digits,
#   so that a number represented as string, i.e., a tuple/word of letters/symbols/characters.
# We allow the user to tell us what to use as digits:
#   To this end, the user should give us a sequence/tuple/list
#   of his/her choice of digits in increasing order from least to greatest digit.
# If the user does not define the digits,
#   we default to the normal decimal digits: "0", "1", ..., "9"
# NB: inden(ta)tion tells Python which instructions form a "block (group)".
#   This is quite natural to indicate which parts of a text form a group;
#   but most programming languages use brackets for this, recall "I saw a man with a telescope on a hill."
def incN(number: str, digits=None) -> str:
  # If the user does not define the digits, we use the canonical decimal digits
  digits = digits or tuple("0123456789")  # let Python unpack the string into a sequence of symbols
  # If the input is the empty string/word,
  if number == "":
    # we treat it as "0" and thus directly return "1"
    return digits[1]  # done
  # Split the number into last and the remaining leading digits:
  leading_digits, last_digit = number[:-1], number[-1]
  # If the last digit is the greatest digit (the last in the sequence),
  if last_digit == digits[-1]:
    # we need to increment the remaining leading digits and append the first/least digit
     return incN(leading_digits, digits) + digits[0]  # done
  # Else we only need to replace the last digit by its successor in the digit sequence
  # Ask Python to find the index (value) of the last digit
  idx = digits.index(last_digit)
  # If we cannot find the last digit within the allowed digits,
  if idx < 0:
    # we get angry and crash the program to tell the user to pay more attention next time!
    assert False, f"{last_digit} is not an admissible digit, stupid!"
  return leading_digits + digits[idx+1]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
# This is essentially `incN` "revesered".
# It would probably be goood to assert that `digits` does not list some symbol twice.
def decN(number: str, digits=None) -> str:
  # If the user does not define the digits, we use the canonical decimal digits
  digits = digits or tuple("0123456789")  # let Python unpack the string into a sequence of symbols
  # The empty string is treated as "0" again.
  if number == "":
    # but decrementing "0" is undefined, so we return None
    return None
  # Split the number again into last and the remaining digits
  leading_digits, last_digit = number[:-1], number[-1]
  # If the last digit is the least digit,
  if last_digit == digits[0]:
    # We need to try to decrement the leading digits and append the greatest digit to the result
    try:
      return decN(leading_digits, digits) + digits[-1]
    except TypeError: # would need to decrement the empty string
      # but if there are no leading digits which we can decrement, we return None again.
      return None
  # else we find the index/value of the last digit and replace it by its predecessor
  idx = digits.index(last_digit)
  # Insult the user in case of a nonadmissible digit.
  assert idx > 0, f"{last_digit} is not an admissible digit, stupid!"
  # Optional: drop leading zeros (literally! so zero is the empty string again)
  # if idx == 1 and leading_digits == "":
  #   return ""
  return leading_digits + digits[idx-1]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def iszeroN(a: str, digits=None) -> bool:
  return decN(a, digits) is None

def cmpN(a: str, b: str, digits=None) -> bool:
  while not iszeroN(a, digits) and not iszeroN(b, digits):
    a = decN(a, digits)
    b = decN(b, digits)
  if iszeroN(a, digits):
    return '=' if iszeroN(b, digits) else '<'
  else:
    return '>'
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def addN(a: str, b: str, digits=None) -> str:      # "official" recursive/inductive definition
  # test if `b` is zero
  b = decN(b, digits)
  # if `b` was zero, `a` is the result, else compute `(a+1)+(b-1)` recursively
  return a if b is None else addN(incN(a, digits), b, digits)

def subN(a: str, b: str, digits=None) -> str:      # "official" recursive/inductive definition
  b = decN(b, digits)                              # test if `b` is zero
  if b is None: return a                           # if `b` was zero, `a` is the result
  a = decN(a, digits)                              # if decN succeeds: `(a-1) - (b-1)`
  return None if a is None else subN(a, b, digits) # if decN failed, no result, else recursion

def convertN(a: str, src_digits, dst_digits) -> str:
  b = ""
  while True:
    a = decN(a, src_digits)
    if a is None: return b
    b = incN(b, dst_digits)

def mulN(a: str, b: str, digits=None) -> str:
  res = ""
  while True:
    b = decN(b, digits)
    if b is None: return res
    res = addN(res, a, digits)

def mulN_recursive(a: str, b: str, digits=None) -> str: # "official" recursive/inductive definition
  b = decN(b)
  return "" if b is None else addN(mulN_recursive(a, b, digits), a, digits)
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def divmodN_iterative(a: str, b: str, digits=None) -> tuple[str, str]:
  s = ""
  while True:
    a_new = subN(a, b, digits)
    if a_new is None: return s, a
    a = a_new
    s = incN(s, digits)
  pass

def divmodN_recursive(a: str, b: str, digits=None) -> tuple[str,str]:
  assert not iszeroN(b, digits)
  a_new = subN(a, b, digits)
  if a_new is None: return '', a
  s, r = divmodN_recursive(a_new, b, digits)
  return incN(s, digits), r

def divmodN_tail_recursive(a: str, b: str, digits=None, s: str="") -> tuple[str,str]:
  assert not iszeroN(b, digits)
  a_new = subN(a, b, digits)
  if a_new is None: return s, a
  return divmodN_tail_recursive(a_new, b, digits, incN(s, digits))

divmodN = divmodN_tail_recursive  # choose any

# For convenience:
def divN(a: str, b: str, digits=None) -> str:
  return divmodN(a,b,digits)[0]

def modN(a: str, b: str, digits=None) -> str:
  return divmodN(a,b,digits)[1]
```

For personal use only, no commercial use. IUAC CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
# Alternatively first unpack: ap, am = a; bp, bm = b
def addZ(a: Z, b: Z, digits=None) -> Z:
  return addN(a[0],b[0],digits), addN(a[1],b[1],digits)

def mulZ(a: Z, b: Z, digits=None) -> Z:
  return addN(mulN(a[0],b[0],digits), mulN(a[1],b[1],digits), digits), \
         addN(mulN(a[0],b[1],digits), mulN(a[1],b[0],digits))

# TODO:
def negZ(a: Z) -> Z:
  return (a[1],a[0])

def subZ(a: Z, b: Z, digits=None) -> Z:
  return addZ(a,negZ(b),digits)

def iszeroZ(a: Z, digits=None) -> bool:
  return cmpN(a[0],a[1],digits) == '='
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def reduceZ(a: Z, digits=None) -> Z:
  ap, am = a
  while not iszeroN(ap, digits) and not iszeroN(am, digits):
    ap = decN(ap, digits)
    am = decN(am, digits)
  return (ap, am)

def signabsZ(a: Z, digits=None) -> tuple[str,N]:
  a = reduceZ(a, digits)
  ap, am = a          # (a_+, a_-)
  c = cmpN(ap, am, digits)
  if c == '<':        # a_+ < a_-
    return ('-', am)
  if c == '>':        # a_+ > a_-
    return ('+', ap)
  return ('0', '')    # a_+ = a_-
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def addQ(a: Q, b: Q, digits=None) -> Q:
  return addZ(mulZ(a[0], b[1], digits), mulZ(a[1], b[0], digits)), mulZ(a[1], b[1], digits)

def mulQ(a: Q, b: Q, digits=None) -> Z:
  # this does not check if we divide by 0
  return mulZ(a[0], b[0], digits), mulZ(a[1], b[1], digits)


# TODO:
def iszeroQ(a: Q, digits=None):
  return iszeroZ(a[0], digits) and not iszeroZ(a[1], digits)

def negQ(a: Q) -> Q:
  return (negZ(a[0],a[1]))

def subQ(a: Q, b: Q, digits=None) -> Q:
  return addQ(a, negQ(b), digits)

def invQ(a: Q) -> Q:
  return (a[1], a[0])

def divQ(a: Q, b: Q, digits=None) -> Q:
  return mulQ(a, invQ(b), digits)
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def gcdN_recursively(a: N, b: N, digits=None) -> str:
  if iszeroN(a, digits):
    return b
  return gcdN_recursively(modN(b,a,digits), a, digits)


def gcdN_iteratively(a: N, b: N, digits=None) -> str:
  while not iszeroN(a, digits):
    a, b = modN(b,a,digits), a
  return b

gcdN = gcdN_recursively

def reduceQ(a: Q, digits=None) -> Q:
  n, d = a  # n, d are integers
  sign_n, abs_n = signabsZ(n, digits)
  sign_d, abs_d = signabsZ(d, digits)
  g = gcdN(abs_n, abs_d, digits)
  abs_n = divN(abs_n, g, digits)
  abs_d = divN(abs_d, g, digits)
  if sign_n == sign_d:
    return ((abs_n,''), (abs_d,''))
  return (('',abs_n), (abs_d,''))
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```python
def prime_factorization(n: int) -> list[tuple[int,int]]:
  res = []

  def _nu(p,n):
    # compute ν_p(n) := max{e ∈ ℕ_0 | n mod p^e = 0}
    e = 0
    k, r = divmod(n,p)
    while r == 0:
      e += 1
      n = k
      k, r = divmod(n,p)
    if e > 0:  # Add p^e to factorization, continue with n/p^e
      res.append((p,e))
    return n


  n = _nu(2, n)           # remove first maximal power of 2
  d = 3                   # only odd divisors remain: 3,5,7, ...
  while d ** 2 <= n:      # If n is not prime, it has a odd divisor d ≤ √n
    n = _nu(d,n)          # If d divides n, it is the least divisor of n and thus prime
    d += 2                # Consider next odd number as potential (prime) divisor

  if n > 1:               # n > 1 has no divisors ≤ √n, thus it is itself prime
    res.append((n,1))

  return res
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def fp(p: int, q: int, b: int, l: int=0) -> str:
  assert 0 <= p < q and b > 1   # i.e. 0 ≤ p/q < 1
  res = '0.'
  for i in range(l):     # Compute the first l digits of p/q = 0.d_1 d_2 ... d_l ... w.r.t. base b
    p = p * b            # before 0 ≤ p/q < 1; after 0 ≤ p/q < b.
    d, p = divmod(p,q)   # d := ⌊p/q⌋ ∈ Z_b, p := p − dq = (b mod q) ∈ Z_q
    res += str(d)
  return res
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```python
def poly_eval_horner_v1(a: Coeffs, x: Number) -> Number:  # OPTIONAL: overwrites given definition
  # a = (a_0, a_1, ..., a_d) ≡ a_0 + ... + a_d X^d = a_0 + X(a_1 + ... + a_d X^{d-1})
  # Recursively evaluate a_1 + ... + a_d X^{d-1}
  if len(a) == 0: return 0
  res = a[-1]
  for ai in a[-2::-1]:
    res = ai + x * res
  return res



from functools import reduce
def poly_eval_horner_v2(a: Coeffs, x: Number) -> Number:
  # `reduce` does exactly the above
  return reduce(lambda p, ai: p*x+ai, reversed(a), 0)

poly_eval = poly_eval_horner_v2



def poly_str(a: Coeffs) -> str:  # OPTIONAL: overwrites given definition
  res = '+'.join(f'{ai}X^{i}' for i, ai in enumerate(a) if ai != 0)
  if res == '': return '0'
  # The lazy way
  res = res.replace('+-', '-').replace('X^0','').replace('+1X','+X').replace('-1X','-X')
  return res
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def poly_bisection(a: Coeffs, xmin: Number, xmax: Number, eps=1e-9) -> Optional[Number]:
  xl = xmin                  # lower
  xu = xmax                  # upper
  yl = poly_eval(a, xl)
  yu = poly_eval(a, xu)
  if yl > yu:                # we swap so that y_l = a(x_l) ≤ a(x_u) = y_u
    yl, yu = yu, yl
    xl, xu = xu, xl
  while yl < 0 < yu:
    xm = (xl+xu)/2           # x_m = (x_l+x_u)/2 midpoint
    if abs(xl-xu) < eps:     # if too close for float, stop
      return xm
    ym = poly_eval(a, xm)    # y_m = a(x_m)
    if ym > 0:
      xu, yu = xm, ym
    else:
      xl, yl = xm, ym
  if yl == 0: return xl
  if yu == 0: return xu
  return None  # no intersection with y = 0 indicated
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```python
# Simple version
def poly_add(a: Coeffs, b: Coeffs) -> Coeffs:
  n = poly_deg(a)
  m = poly_deg(b)
  if n < m:
    a, b = b, a
    n, m = m, n
  res = [a[i] for i in range(n+1)]  # result will have degree at most n
  for i in range(m+1):
    res[i] += b[i]
  return tuple(res)  # N.B.: if you remove trailing 0s, check that `poly_divmod` still is correct

# Short
from itertools import product, zip_longest
def poly_add(a: Coeffs, b: Coeffs):
  # extend sequences to same length by appending sufficiently many 0s to the shorter one
  # N.B.: if you remove trailing 0s, check that `poly_divmod` still is correct
  return tuple(ai + bi for ai, bi in zip_longest(a, b, fillvalue=0))
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```python
# Simple version
def poly_mul(a: Coeffs, b: Coeffs) -> Coeffs:
  n = poly_deg(a)
  m = poly_deg(b)
  if m < 0 or n < 0: return ()  # the zero polynomial
  res = list(0 for _ in range(m+n+1))  # result will have degree n*m
  for k in range(m+n+1):
    for i in range(k+1):
      ai  = a[i] if i <= n else 0
      bki = b[k-i] if 0 <= k-i <= m else 0
      res[k] += ai * bki
  return tuple(res)                    # no need to reduce as long as no zero divisors

# Short
def poly_mul(a: Coeffs, b: Coeffs) -> Coeffs:
  n = poly_deg(a)
  m = poly_deg(b)
  if m < 0 or n < 0: return ()  # the zero polynomial
  # Cauchy product: (a_0 + ... + a_n X^n)(b_0 + ... + b_m X^m) = sum_{k=0}^{m+n} X^k sum_{i=0} a_i b_{k-i}
  # N.B.: `range(N)=(0,1,...,N-1)`
  # N.B.: a_i only defined for 0 <= i <= n i.e. 0 <= i < n+1
  # N.B.: b_{k-i} only defined for 0 <= k-i <= m i.e. -k <= -i <= m-k i.e. k >= i >= k-m i.e. k-m <= i < k+1
  return tuple(sum(a[i]*b[k-i] for i in range(max(0,k-m),min(n,k)+1)) for k in range(m+n+1))
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```python
from fractions import Fraction

# optional
def poly_int_to_frac(a: Coeffs) -> bool:
  if all(isinstance(ai, int) for ai in a):
    return tuple(Fraction(a[i]) for i in range(poly_deg(a)+1))
  return a

def poly_frac_to_int(a: Coeffs) -> Coeffs:
  if all(isinstance(ai, Fraction) and abs(ai.denominator) == 1 for ai in a):
    return tuple(int(a[i]) for i in range(poly_deg(a)+1))
  return a

def poly_divmod_iterative(a: Coeffs, b: Coeffs) -> Tuple[Coeffs, Coeffs]:
  db = poly_deg(b)
  assert db >= 0
  a, b = poly_int_to_frac(a), poly_int_to_frac(b)  # optional
  s = ()
  r = a
  while (dr := poly_deg(r)) >= db:                 # Invariant: a = r + b·s  (':=' Walrus operator in Python)
    t = [0 for _ in range(dr-db)] + [r[dr]/b[db]]  # t := LC(r)/LC(b) X^(deg(r)-deg(b))
    s = poly_add(s, t)                             # s := s + t  (afterwards: r + b·s = a + tb)
    t[-1] = -t[-1]                                 # t := -t
    r = poly_add(r, poly_mul(b, t))                # r := r - b·t  (aftwards: r + b·s = a again)
  r = r[:dr+1]
  s, r = poly_frac_to_int(s), poly_frac_to_int(r)  # optional
  return s, r
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```python
def poly_divmod_recursive(a: Coeffs, b: Coeffs) -> tuple[Coeffs, Coeffs]:
  b = b[:poly_deg(b)+1]
  assert len(b) > 0
  a = a[:poly_deg(a)+1]
  if len(a) < len(b):
    return (), a
  lc_a = a[-1]
  lc_b = b[-1]
  c = lc_a / lc_b
  if isinstance(lc_a, int) and isinstance(lc_b, int):  # optional
    c = Fraction(lc_a,lc_b)
    if c.denominator == 1: c = int(c)
  t = [0 for _ in range(len(a)-len(b))] + [-c]          # t := -\frac{LC(a)}{LC(b)} X^{deg(a)-deg(b)}
  a_new = poly_add(a, poly_mul(b, t))                  # a' := a + t * b
  s, r = poly_divmod_recursive(a_new, b)               # a' = s * b + r with deg(r) < deg(b)
  t[-1] = - t[-1]                                      # t := -t
  s = poly_add(s, t)                                   # s := s + t
  return s, r
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def poly_divmod_tail_recursive(a: Coeffs, b: Coeffs, s: Optional[Coeffs]=None) -> tuple[Coeffs, Coeffs]:
  # This already computes `s` on the way down to the induction basis which can be more efficient.
  b = b[:poly_deg(b)+1]
  assert len(b) > 0
  s = s or ()
  a = a[:poly_deg(a)+1]
  if len(a) < len(b):
    return s, a
  lc_a = a[-1]
  lc_b = b[-1]
  c = lc_a / lc_b
  if isinstance(lc_a, int) and isinstance(lc_b, int):  # optional
    c = Fraction(lc_a,lc_b)
    if c.denominator == 1: c = int(c)
  t = [0 for _ in range(len(a)-len(b))] + [-c]          # t := -LC(a)/LC(b) X^(deg(a)-deg(b))
  a_new = poly_add(a, poly_mul(b, t))                  # a' := a + t · b
  t[-1] = - t[-1]                                      # t := -t
  s = poly_add(s, t)                                   # s := s + t
  return poly_divmod_tail_recursive(a_new, b, s)       # a' = s · b + r with deg(r) < deg(b)
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
poly_divmod = poly_divmod_tail_recursive  # does not matter, just choose one

# For convenience
def poly_div(a: Coeffs, b: Coeffs) -> Coeffs:
  return poly_divmod(a,b)[0]

def poly_mod(a: Coeffs, b: Coeffs) -> Coeffs:
  return poly_divmod(a,b)[1]

# GCD iterative
def poly_gcd_iterative(a: Coeffs, b: Coeffs) -> Coeffs:
  while poly_deg(a) >= 0:
    a, b = poly_mod(b,a), a
  return b

# GCD recursive
def poly_gcd_recursive(a: Coeffs, b: Coeffs) -> Coeffs:
  if poly_deg(a) < 0:
    return b
  return poly_gcd(poly_mod(b,a), a)

poly_gcd = poly_gcd_recursive  # choose one
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def poly_newton(a: Coeffs, x0: Number, eps=1e-9, maxsteps=100) -> Optional[Number]:
  da = poly_diff(a)
  g = poly_gcd(a,da)
  a = poly_div(a,g)
  da = poly_div(da,g)
  for i in range(maxsteps):
    y0 = poly_eval(a,x0)
    if abs(y0) < eps:
      return x0
    x0 = x0 - y0 / poly_eval(da,x0)
  return None
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
# Same as for rational numbers

def pf_add(a: PF, b: PF) -> PF:
  return  poly_add(poly_mul(a[0],b[1]),poly_mul(a[1],b[0])), poly_mul(a[1],b[1])

def pf_mul(a: PF, b: PF) -> PF:
  return  poly_mul(a[0],b[0]), poly_mul(a[1],b[1])

def pf_cancel(a: PF) -> PF:
  g = poly_gcd(a[0],a[1])
  return poly_div(a[0],g), poly_div(a[1],g)


def poly_diff(a: Coeffs) -> Coeffs:
  return tuple(ai*(i+1) for i, ai in enumerate(a[1:]))

# Reminder: (p/q)' = (p'/q) - (p/q) * (q'/q)
def pf_diff(a: PF) -> PF:
  p, q = a
  dp = poly_diff(p)
  dq = poly_diff(q)
  return pf_add((dp,q), pf_mul(a,(poly_neg(dq),q)))
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def pf_find_local_extremum(f: PF, xl: Number, xu: Number, eps=1e-9) -> Optional[Number]:
  print(f"f={pf_str(f)}")
  f = pf_cancel(f)
  print(f"f={pf_str(f)} after canceling")
  df = pf_diff(f)
  print(f"f'={pf_str(df)} (first derivative)")
  if xl > xu: xu, xl = xl, xu

  x0 = poly_bisection(df[0],xl,xu,eps=eps)
  if x0 is None:
    print(f"No root of first derivative found in [{xl},{xu}]")
    return
  print(f"Found root of first derivative: x0={x0}, f'(x0)={pf_eval(df,x0)}")
  k = 1
  dkf = df
  X_x0 = (-x0,1)
  taylor = X_x0
  while abs(pf_eval(dkf,x0)) < eps:
    dkf = pf_diff(dkf)
    k += 1
    taylor = poly_mul(taylor, X_x0)
  taylor = poly_mul(taylor,(pf_eval(dkf,x0)/math.factorial(k),))
  taylor = poly_add(taylor,(pf_eval(f,x0),))
  print(f"Taylor polynomial at x0: {poly_str(taylor)}")
  pf_plot([f,df,(taylor,(1,))], xl, xu)
  plt.show()
  return x0
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def __sub__(self, other: 'Vc') -> 'Vc':
  return Vc(self.x-other.x, self.y-other.y)

def __truediv__(self, other: Scalar) -> 'Vc':
  return self * (1/other)

def __rmul__(self, other: Scalar) -> 'Vc':
  return self * other

# ---

def snorm(self) -> Scalar:
  return self.x ** 2 + self.y ** 2

def norm(self) -> Scalar:
  return math.sqrt(self.snorm())

# ---

def perp(self) -> 'Vc':
  return Vc(-self.y,self.x)

def __matmul__(self, other: 'Vc') -> Scalar:
  return self.x * other.x + self.y * other.y
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
class Cr(NamedTuple):
  c: Vc
  p: Vc

  def plot(self, ax, shape='-', n: int=100):
    alpha = 2*math.pi/n
    r = math.sqrt((self.p-self.c).snorm())
    xs = [self.c.x + r * math.cos(alpha*i) for i in range(n+1)]
    ys = [self.c.y + r * math.sin(alpha*i) for i in range(n+1)]
    ax.plot(xs,ys,shape,color="green",linewidth=1)

  pass  # end of class
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def intersect_lines(l1: Ln, l2: Ln) -> Vc:
  n, delta = line_hnf(l1)
  c, d = l2
  return [c + lam * (d-c) for lam in solve_linear(n @ (d-c), (n@c)-delta)]

def intersect_line_circle(ln: Ln, cr: Cr) -> list[Vc]:
  a, b = ln
  c, d = cr
  return [a+lam*(b-a) for lam in solve_quadractic((b-a).snorm(),-2*((b-a)@(c-a)),(c-a).snorm()-(c-d).snorm())]

def intersect_circles(c1: Cr, c2: Cr) -> list[Vc]:
  a, b = c1
  c, d = c2
  srho1 = (b-a).snorm()
  srho2 = (d-c).snorm()
  if srho1 > srho2:
    srho1, srho2 = srho2, srho1
    a, c = c, a
  v = c-a
  sdelta = v.snorm()
  lam = (srho1 - srho2)/(2*sdelta) + 0.5
  smu = (srho1 - lam**2 * sdelta)/sdelta
  if smu < 0:
    return []
  p = a + lam * v
  d = v.perp() * math.sqrt(smu)
  return [p-d,p+d]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

4 Solutions

- $\mathbb{N}$
- $\mathbb{Z}$
- $\mathbb{Q}$
- Primes
- Fractional part
- Polynomials
- Rational fractions
- Analytic geometry in the Euclidean plane
- Analytic geometry in the Euclidean space (*)

For personal use only, no commercial use – TUM CIT Department of Computer Science

- In the following vectors just denoted by a, b, c, d, v, w, x, y.

  Scalars are denoted by $\delta, \lambda, \mu, \rho$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Points of minimal distance between two lines in $\mathbb{R}^3$

- First orthogonal projection of a point c onto the line $a + \lambda v$:

$$p = a + \frac{\langle v, c-a \rangle_2}{\|v\|_2^2} v$$

Applying Pythagoras to the right triangle defined by a, c, p:

$$\|c-p\|_2^2 = \|c-a\|_2^2 - \|p-a\|_2^2 = \|c-a\|_2^2 - \frac{\langle v, c-a \rangle_2^2}{\|v\|_2^2}$$

Second, replace c by $c + \mu w$ to obtain:

$$\|c+\mu w-p\|_2^2 = \|c-a+\mu w\|_2^2 - \frac{\langle v, c-a+\mu w \rangle_2^2}{\|v\|_2^2}$$

Expanding:

$$\|c-a\|_2^2 + 2\mu\langle c-a, w\rangle_2 + \mu^2\|w\|_2^2 - \frac{\langle v,c-a\rangle_2^2 + 2\mu\langle v,c-a\rangle_2\langle v,w\rangle_2 + \mu^2\langle v,w\rangle_2^2}{\|v\|_2^2}$$

Derivative w.r.t. μ:

$$2\mu\|w\|_2^2 + 2\langle w, c-a\rangle_2 - 2\mu\frac{\langle v,w\rangle_2^2}{\|v\|_2^2} - 2\frac{\langle v,w\rangle_2\langle v,c-a\rangle_2}{\|v\|_2^2} \stackrel{!}{=} 0$$

$$\mu = -\frac{\langle w,c-a\rangle_2\|v\|_2^2 - \langle v,w\rangle_2\langle v,c-a\rangle_2}{\|w\|_2^2\|v\|_2^2 - \langle v,w\rangle_2^2}$$

Symmetrically, i.e. by swapping the roles of $a + \lambda v$ and $c + \mu w$:

$$\lambda = \frac{\langle v,c-a\rangle_2\|w\|_2^2 - \langle v,w\rangle_2\langle w,c-a\rangle_2}{\|v\|_2^2\|w\|_2^2 - \langle v,w\rangle_2^2}$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

Intersection: line-plane, line-sphere $\mathbb{R}^3$

- The cross product (vector product) $a \times b$ of two vectors in $\mathbb{R}^3$ yields a vector which is orthogonal to both (assuming a, b are not linearly dependent).

  Given a plane defined by a, b, c, we thus obtain a Hesse normal form $\langle n, x \rangle_2 + \delta = 0$ by choosing $n = a \times b$ and $\delta = -\langle n, a \rangle_2$.

- Given a plane in HNF and a line in parametrized form $a + \lambda v$ simply subsitute the latter into the former: $\langle n, a + \lambda v \rangle_2 + \delta = 0$.

  If $\langle n, v \rangle_2 \neq 0$, we obtain $\lambda = -\frac{\delta + \langle n, a \rangle_2}{\langle n, v \rangle_2}$.

- To intersect a line $a + \lambda v$ with a sphere $\|x - c\|_2^2 = \rho^2$ again, simply substitute and solve the resulting quadratic equation (if possible):

  $\|a + \lambda v - c\|_2^2 = \rho^2$ **iff** $\|a - c\|_2^2 - \rho^2 + 2\lambda \langle a - c, v \rangle_2 + \lambda^2 \|v\|_2^2 = 0$

- Reminder: from $\|x\|_2^2 = \langle x, x \rangle_2$ it follows that

$$\|\lambda x + \mu y\|_2^2 = \lambda^2 \|x\|_2^2 + 2\lambda\mu \langle x, y \rangle_2 + \mu^2 \|y\|_2^2$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def intersect_line_plane_3d(ln: Ln3, pl: Pl3):
  n, delta = plane_hnf(pl)
  a, b = ln
  v = b-a
  try:
    lam = - (delta+(n@a))/(n@v)
  except ZeroDivisionError:
    return []
  return [a+v*lam]

def intersect_line_sphere_3d(ln: Ln3, sp: Sp3):
  c, p = sp
  a, b = ln
  alpha = (b-a).snorm()
  beta = 2 * (a-c) @ (b-a)
  gamma = (a-c).snorm() - (c-p).snorm()
  try:
    D = math.sqrt(beta**2 - 4*alpha*gamma)
    sol = [(-beta - D)/(2*alpha), (-beta + D)/(2*alpha)]
  except ArithmeticError:
    return []
  return [a+(b-a)*lam for lam in sol]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

Intersection: plane-plane $\mathbb{R}^3$

- E.g. convert one plane into HNF $\langle n, x\rangle_2 + \delta = 0$, parametrize the other $a + \lambda v + \mu w$.

  Substitution leads to

  $$\langle n, a + \lambda v + \mu w\rangle_2 + \delta = 0$$

  iff

  $$\langle n, a\rangle_2 + \lambda\langle n, v\rangle_2 + \mu\langle n, w\rangle_2 + \delta = 0$$

  Assuming $\langle n, v\rangle_2 \neq 0$ (else swap v, w; if still 0, either no solution or planes coincide)

  $$\lambda = -\frac{\delta + \langle n, a\rangle_2 + \mu\langle n, w\rangle_2}{\langle n, v\rangle_2}$$

  In general, the intersection is a line; e.g. if $\langle n, v\rangle_2 \neq 0$, we can choose $\mu = 0$ and $\mu = \langle n, v\rangle_2$ and λ accordingly, to obtain two points on this line.

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def intersect_planes_3d(p1: Pl3, p2: Pl3):
  n, delta = plane_hnf(p1)
  a, b, c = p2
  v = b-a
  w = c-a
  mu = n @ v
  if mu == 0:
    v, w = w, v
    mu = n @ v
  try:
    lam1 = - (delta+(n@a))/mu
    lam2 = - lam1 - (n @ w)
  except ZeroDivisionError:
    return []
  return [Ln3(a+v*lam1,a+v*lam2+w*mu)]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

Intersection: plane-sphere $\mathbb{R}^3$

- Convert the plane into HNF $\langle n, x \rangle_2 + \delta = 0$.

  Compute the projection of the center c of the sphere onto the plane, i.e. intersect the plane with $c + \lambda n$ to obtain the point c'.

  If the distance of the plane to the sphere is at most the radius of the plane, the intersection will be circle in the plane with center c'; its radius can be computed using Pythogoras again.

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def intersect_plane_sphere_3d(pl: Pl3, sp: Sp3):
  c, p = sp
  n, delta = plane_hnf(pl)
  n = n / n.norm()
  srho = (p-c).snorm()
  # <n,c + lam * n> + delta = 0 IFF lam = - (delta - <n,c>)
  lam = (n@c -delta)
  sdist = lam ** 2
  if sdist > srho:
    return []
  c2 = c + n * lam
  rho2 = math.sqrt(srho - sdist)
  p2 = n ^ Vc3(1,0,0)
  if p2.iszero():
    p2 = n ^ Vc3(0,1,0)
  return [Cr3(c2, c2+p2*rho2, n)]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

- If the spheres intersect, their intersection is a circle which is located in a plane $\langle n, x - p \rangle_2 = 0$ with p the intersection of the plane with the line connecting the centers of the circles.

  p can be computed using Pythagoras just as in the case of two circles in the Euclidean plane.

  Afterwards, intersect the plane with one of the spheres (if there is an intersection).

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
def intersection_spheres_3d(sp1: Sp3, sp2: Sp3):
  # essentially the same code as for the circle
  a, b = sp1
  c, d = sp2
  srho1 = (b-a).snorm()
  srho2 = (d-c).snorm()
  if srho1 > srho2:
    srho1, srho2 = srho2, srho1
    a, c = c, a
  n = c-a
  # we normalize the vector to simplify the expressions
  n = n / n.norm()
  lam = srho1 - srho2 + 0.5
  smu = srho1 - lam**2
  if smu < 0:
    return []
  # the same as for intersecting the plane defined by n and c2 with the smaller sphere
  # `smu` is `sdist` from above
  c2 = a + lam * n
  p2 = n ^ Vc3(1,0,0)
  if p2.iszero():
    p2 = n ^ Vc3(0,1,0)
  return [Cr3(c2, c2+p2*math.sqrt(smu), n)]
```

For personal use only, no commercial use – TUM CIT Department of Computer Science